

Bios 740: Deep Learning Methods for Biomedical Applications with Pytorch

Hongtu Zhu, Ph.D.
Kenan Distinguished Professor

BIOS 740: Bridging Deep Learning & Biomedical Science

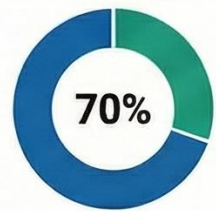
Course at a Glance



Apply Deep Learning to Biomedical Challenges

Use PyTorch to solve real-world problems in medical imaging, genomics, and clinical data.

Grading: 70% Homework, 30% Project



■ Homework
■ Project

Who Is This For?

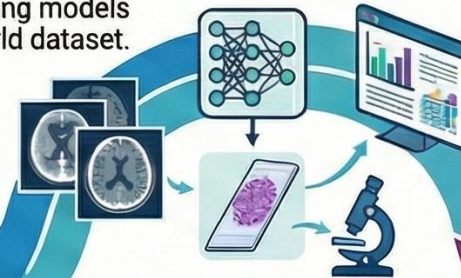


Graduate students with basic Python skills. A math background is a plus.

The Final Project

Application

Apply existing models to a real-world dataset.



Report Focus: 60% on Methods & Experiments

60%

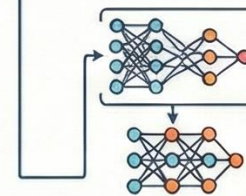
Methods & Experiments

A 4-page report heavily weighted on the details of your model and validation results

Choose Your Path

Model Development

Design a novel deep learning model.



4-Page PDF



PyTorch Code

Deliverable:

4-Page PDF + Code

Submit your formatted report plus supplementary materials like your PyTorch code.

Course Overview

Description: This course provides an in-depth exploration of deep learning methods applied to biomedical data using PyTorch. It covers foundational neural network architectures, advanced models like convolutional neural networks (CNNs), recurrent neural networks (RNNs), graph neural networks (GNNs), generative models, transformers, and foundation models, with applications in biomedical data analysis tasks.

Goals:

- Develop expertise in deep learning methodologies tailored for biomedical applications.
- Gain hands-on experience implementing, training, and evaluating models using PyTorch.
- Tackle real-world challenges through project work and interactive discussions.

Instructor:

Dr. Hongtu Zhu (Email: htzhu@email.unc.edu)

Teaching Assistant:

Mr. Runpeng Dai (Email: runpeng@unc.edu)

Course Format and Structure

- **Schedule:** Tuesdays and Thursdays, 9:30–10:45 AM.
- **Location:** 2301 MCG.
- **Format:** Seminar-style sessions with supplemental readings, interactive discussions, and case studies.
- **Prerequisites:** Basic Python programming; familiarity with linear algebra, probability, and calculus is recommended.
- **Grading:**
 - Homework Assignments: 70% (5 assignments, each worth 15%).
 - Final Project: 30%.
 - No midterms or final exams.
- **Help Sessions:** Weekly help sessions are available to support students with homework, project work, and technical questions related to PyTorch and deep learning concepts.
- **Office Hours:** Friday 12:00pm-1:00pm at 2301 MCG.
- **AI Policy:** Generative AI is allowed, provided its usage is documented.

Course Websites

<https://bios740.github.io/>

BIOS740

Q Search BIOS740

BigS2 LabUNC Biostatistics

About

Syllabus

Staff

Calendar

Short Courses

Install Jeckyll

Welcome to BIOS740

 Welcome to BIOS 740 – Spring 2026 Offering! 

November 8, 2025

We are delighted to welcome new students to the **Spring 2026 offering of BIOS 740: Deep Learning Methods in Biomedical Sciences with PyTorch!**

This marks the **second time** the course is being offered, building on the success and feedback from our inaugural 2025 session.

Throughout the semester, we will explore modern deep learning architectures—including CNNs, RNNs, GANs, and Transformers—and their applications to challenging statistical problems. Stay tuned for updates on lecture materials, coding sessions, and project information. We look forward to another exciting semester of learning and discovery together!

 Join Our Short Course at ENAR 2026! 

November 8, 2025

We are thrilled to announce that we will be offering a short course at the **2026 ENAR Spring Meeting in Indianapolis, Indiana!**

This site uses [Just the Docs](#), a documentation theme for Jeckyll.

<https://bios740.github.io/syllabus/>

BIOS740

Q Search BIOS740

BigS2 LabUNC Biostatistics

About

Syllabus

Staff

Calendar

Short Courses

Install Jeckyll

Syllabus

Fundamentals of Deep Learning

Lec1: [Introduction to Deep Learning, PyTorch & Basic Algorithms](#)

Lec2: [Neural Networks Fundamentals](#)

Basic Network Structures

Lec3: [Convolutional Neural Networks \(CNN\)](#)

HW 1

Lec4: [Recurrent Neural Networks \(RNN\) and Long Short-Term Memory \(LSTM\)](#)

HW 2

HW 1 DUE

Lec5: [Graph Neural Networks: GNN, GCN](#)

HW 3

HW 2 DUE

Final Project

Objective: Apply deep learning concepts and tools to address a biomedical challenge. Projects can follow one of two tracks:

- **Applications Track:** Solve a practical problem using deep learning models.
- **Models Track:** Develop or improve a deep learning model for biomedical tasks, potentially leading to publishable work.

Deliverables:

- **Report:** 4 pages, adhering to the provided template.
- **Submission:** Final project report and supplementary materials (e.g., source code, visualizations).
- **Structure:**
 - Title, Abstract, Introduction, Related Work, Data, Methods, Experiments, Conclusion, Writing/Formatting.

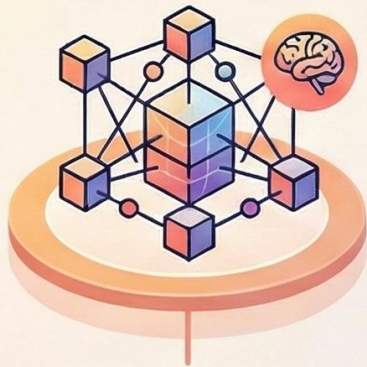
For more details, visit the [course website](#) or [homework repository](#). Let me know if you'd like additional refinements!

Course Grading Scale(s)

- Final course grades will be determined using the following [UNC Graduate School grading scale](#)[Links to an external site.](#). The relative weight of each course component is shown in the Graded Assignments section.
- **H**—High Pass (90-100): Clear excellence
- **P**—Pass (75-90): Entirely satisfactory graduate work
- **L**—Low Pass (60-75): Inadequate graduate work
- **F**—Fail (0-60)

A Journey Through Deep Learning: Course Roadmap

PART 1: FOUNDATIONS & ADVANCED ARCHITECTURES (Sessions 1-8)



Sessions 1-4: Building the Foundation

Kicks off with PyTorch, NNs, CNNs, and RNNs. HW1 is due at session 3.



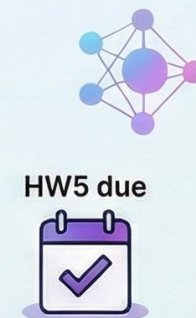
Sessions 5-8: Exploring Advanced Models

Dives into GCNs, GANs, Transformers, and Large Language Models. HW2 and HW3 due.



Sessions 9-12: Focusing on Applications

Covers Image Segmentation, Registration, Spatio-temporal Processes, and Deep Reinforcement Learning. HW4 & HW5 due.



Sessions 13-15: Expert Insights & Final Project

Features guest lectures and culminates with the final project submission.

Bios 740- Chapter 1. Introduction to Deep Learning and Computing Resources

Acknowledgement: Thanks to Miss Jiarui Tang and Mr. Runpeng Dai for preparing some of the slides. We also copied pictures from various websites and friends (e.g., Marc).

Content

1 Introduction to Deep Learning

2 Introduction to PyTorch

3 Introduction to UNC Research Computing Resources

4 Introduction to Basic Algorithm

Content

1 Introduction to Deep Learning

2 Introduction to PyTorch

3 Introduction to UNC Research Computing Resources

4 Introduction to Basic Algorithm

Deep Learning

Deep

Many hidden layers

Learning

Supervised, semi-supervised, unsupervised
Learn adaptive parameters

- Use a cascade of multiple layers of nonlinear processing units for feature extract and transformation
- Learn in supervised and/or unsupervised manner
- Learn representations in different level of abstraction

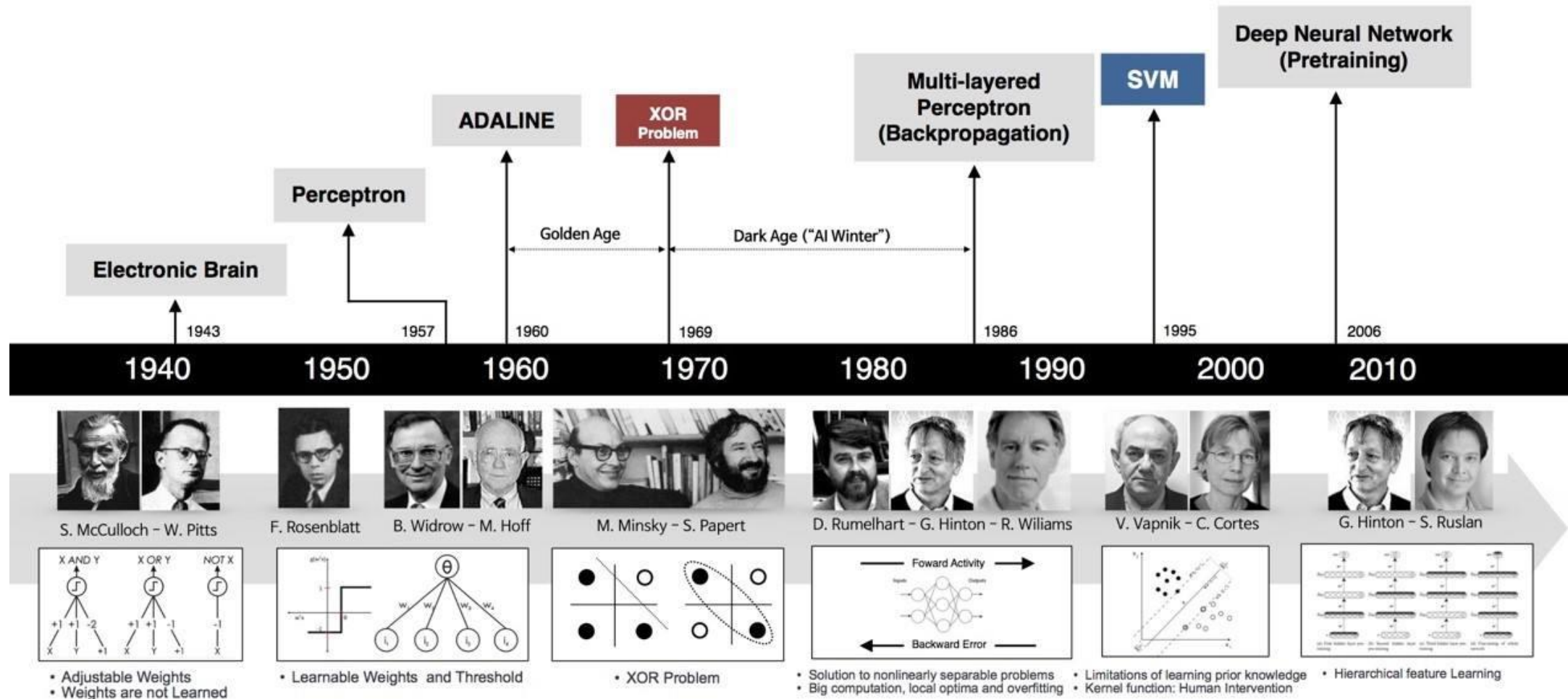
Why popular?

- Chip processing ability
- Increased size of data for training
- Advances in machine learning and signal/information researches



Deep models to efficiently exploit **complex, compositional nonlinear** functions to learn **distributed and hierarchical feature representations**, to make best use

Historical Summary



Deep Learning Explosion



Downloaded from the NSF website and the medium.com

Deep Learning Platforms

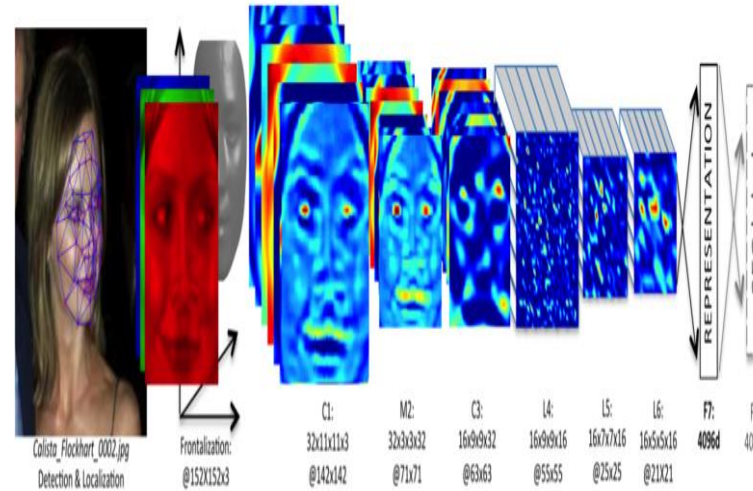


theano



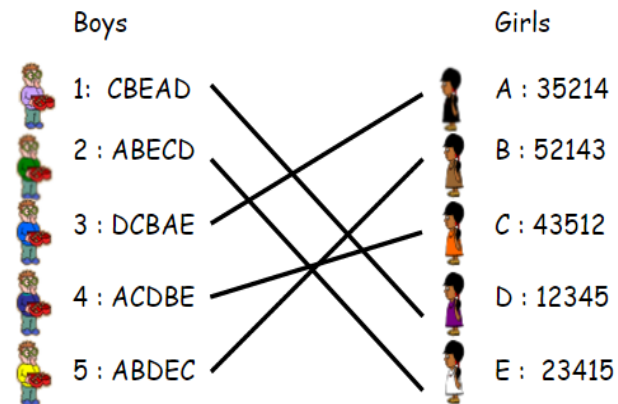
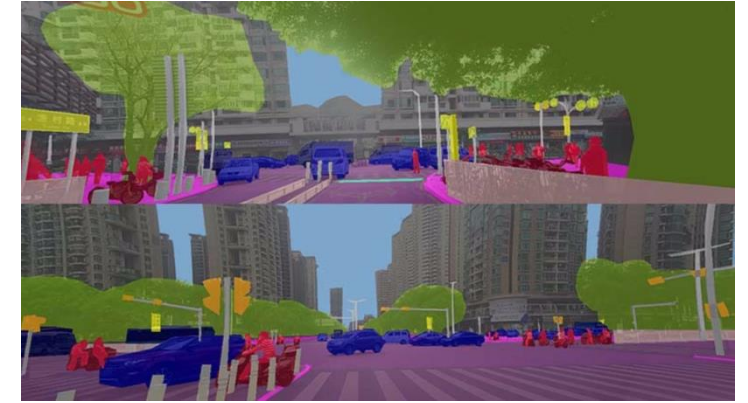
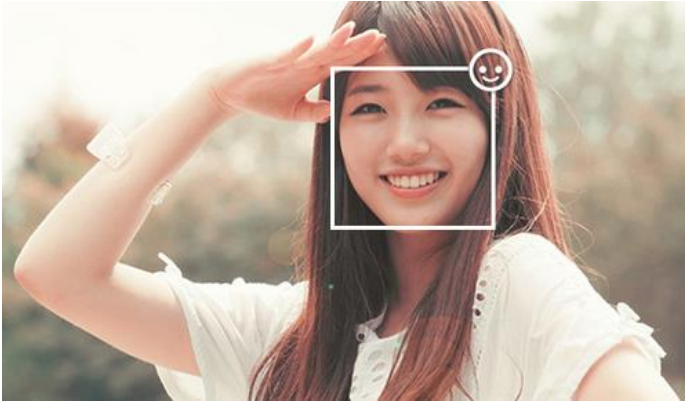
Applications - Vision

IMAGENET



Input	[CLS]	my	dog	is	cute	[SEP]	he	likes	play	##ing	[SEP]
	[MASK]						[MASK]				
Token Embeddings	$E_{[CLS]}$	E_{my}	$E_{[MASK]}$	E_{is}	E_{cute}	$E_{[SEP]}$	E_{he}	$E_{[MASK]}$	E_{play}	$E_{##ing}$	$E_{[SEP]}$
Sentence Embedding	+	+	+	+	+	+	+	+	+	+	+
	E_A	E_A	E_A	E_A	E_A	E_B	E_B	E_B	E_B	E_B	E_B
Transformer Positional Embedding	+	+	+	+	+	+	+	+	+	+	+
	E_0	E_1	E_2	E_3	E_4	E_5	E_6	E_7	E_8	E_9	E_{10}

Applications - Vision



Applications - Vision



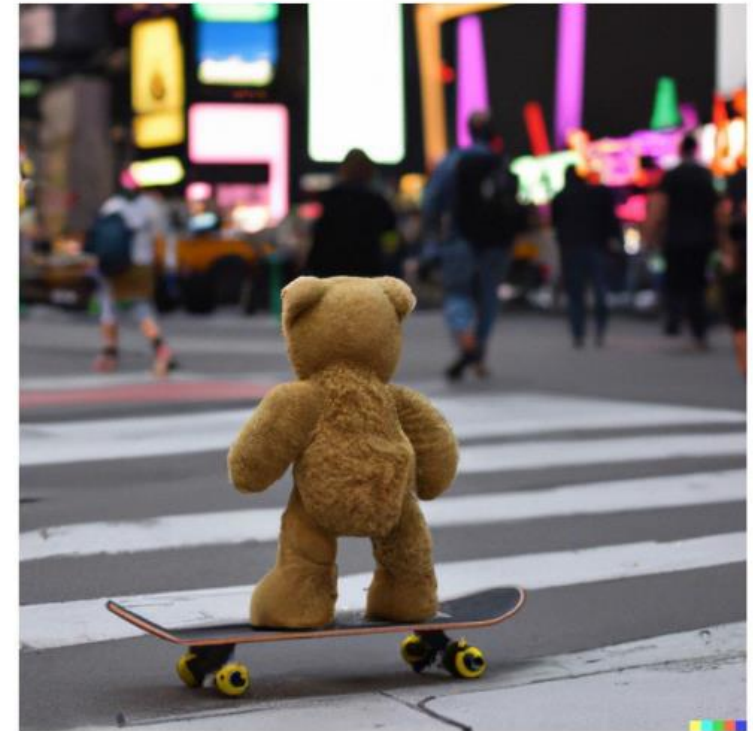
Disease Detection in Healthcare and Medicine

Deep learning can be utilized for early and more accurate detection of diseases like cancer, Alzheimer's, and heart diseases through image analysis.

High quality image generalization

DALL·E 2

“a teddy bear on a skateboard in times square”

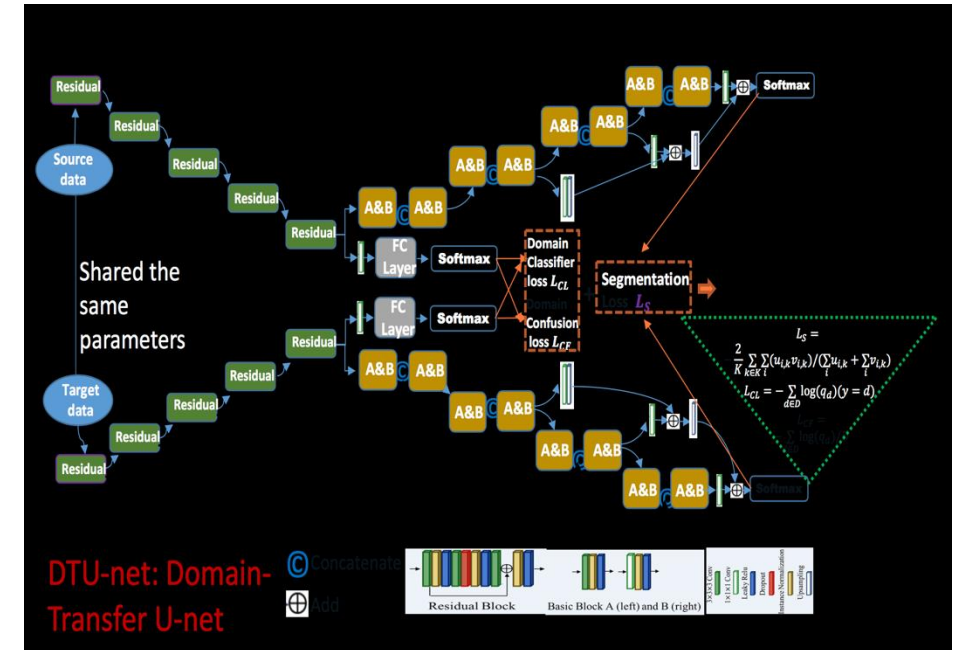


[“Hierarchical Text-Conditional Image Generation with CLIP Latents”](#)
Ramesh et al., 2022

Applications – Medical Imaging

Segmentation Annotation

U-Nets



Liu, Q., Xu, Z., Bertasius, G., & Niethammer, M. (2023). SimpleClick: Interactive Image Segmentation with Simple Vision Transformers. ICCV, 22290-22300. 2023.

Azad *et al.*, “Medical Image Segmentation Review: The success of U-Net.” arXiv, Nov. 27, 2022.

Minaee, Shervin, et al. "Image segmentation using deep learning: A survey." *IEEE PAMI* 44.7 (2021): 3523-3542

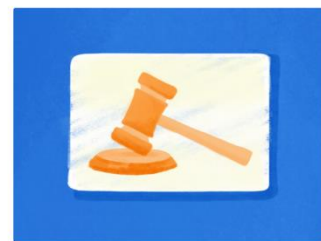
Application - Language

Language Translation in Natural Language Processing

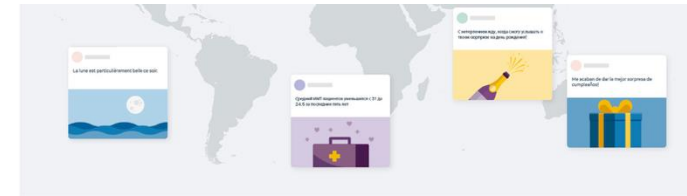
Deep learning enhances real-time, accurate translation of languages, as seen in tools like Google Translate. The following picture shows the translation of a webpage from English to Chinese.



- Facebook AI is introducing M2M-100, the first multilingual machine translation (MMT) model that can translate between any pair of 100 languages without relying on English data. It's open sourced [here](#).
- When translating, say, Chinese to French, most English-centric multilingual models train on Chinese to English and English to French, because English training data is the most widely available. Our model directly trains on Chinese to French data to better preserve meaning. It outperforms English-centric systems by 10 points on the widely used BLEU metric for evaluating machine translations.
- M2M-100 is trained on a total of 2,200 language directions — or 10x more than previous best, English-centric multilingual models. Deploying M2M-100 will improve the quality of translations for billions of people, especially those that speak low-resource languages.
- This milestone is a culmination of years of Facebook AI's foundational work in machine translation. Today, we're sharing details on how we built a more diverse MMT training data set and model for 100 languages. We're also [releasing the model, training, and evaluation setup](#) to help other researchers reproduce and further advance multilingual models.



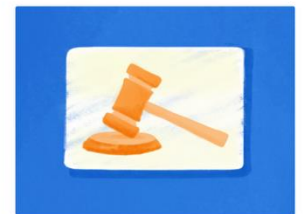
Meta
Meta and Christian Louboutin File
Joint Lawsuit Against Counterfeiter
November 16, 2023



- Facebook AI 正在推出 M2M-100，这是第一个多语言机器翻译 (MMT) 模型，可以在 100 种语言中的任意对之间进行翻译，而无需依赖英语数据。[这里是开源的](#)。
- 例如，在将中文翻译成法语时，大多数以英语为中心的多语言模型都会在中文到英语和英语到法语上进行训练，因为英语训练数据是最广泛可用的。我们的模型直接对中文到法语的数据进行训练，以更好地保留含义。在广泛使用的用于评估机器翻译的 BLEU 指标上，它比以英语为中心的系统高出 10 个点。
- M2M-100 接受了总共 2,200 种语言方向的训练，比以前最好的、以英语为中心的多语言模型多了 10 倍。部署 M2M-100 将为数十亿人提高翻译质量，尤其是那些使用资源匮乏语言的人。
- 这一里程碑是 Facebook AI 多年来在机器翻译领域基础工作的结晶。今天，我们将分享如何为 100 种语言构建更加多样化的 MMT 训练数据集和模型的详细信息。我们还发布了[模型、训练和评估设置](#)，以帮助其他研究人员重现和进一步推进多语言模型。



元
支持独立研究的新工具
2023 年 11 月 21 日

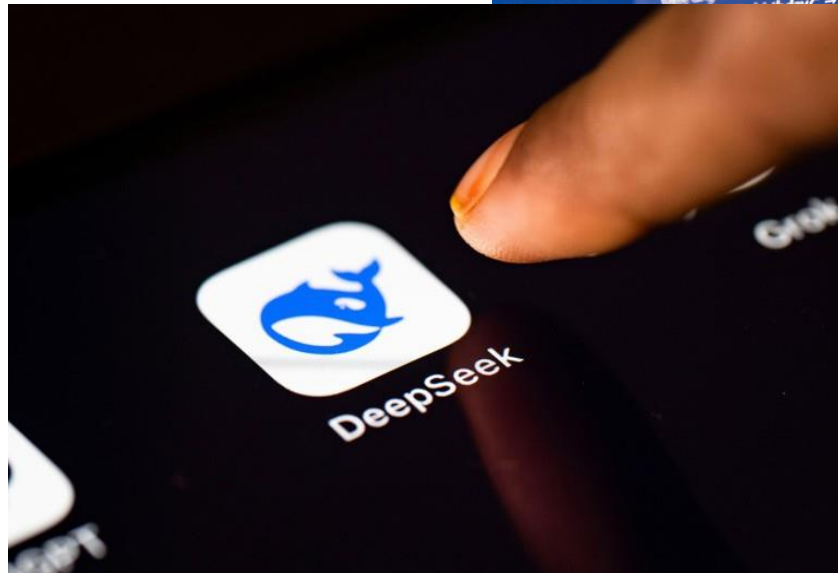


元
Meta 和 Christian Louboutin 对仿冒者提

Application - Language

Large language models

Large language models can perform various tasks such as answering questions, generating creative content, summarizing text, translating languages, and engaging in conversations. It's designed to understand and generate text in a coherent and contextually relevant manner.



<https://www.nature.com/articles/d41586-025-03015-6>

Application - Decision

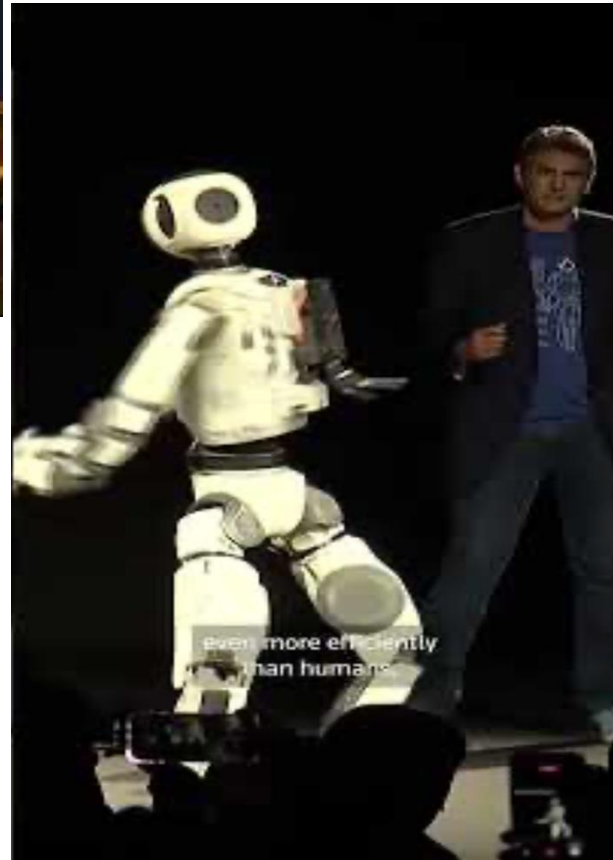


March 2016, AlphaGo made headlines by defeating Lee Sedol.



Reinforcement learning methods have shown priority in video games.

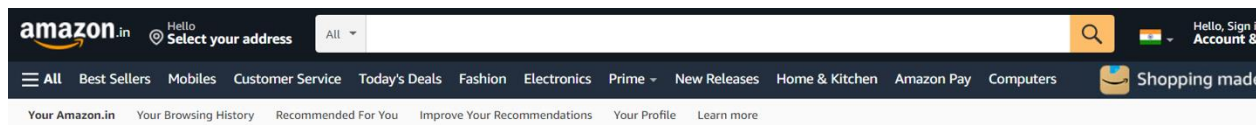
Application – Autonomous Driving & Robotics



Applications - more

Personalized Shopping Experience in Retail and E-Commerce

Deep learning is leveraged to provide personalized recommendations and targeted advertising to customers based on their shopping behavior.



Top picks for you

 boAt Airdopes 141 True Wireless Earbuds with 42H Playtime... ★★★★☆ 58,038 ₹1,499.00 Get it by Saturday, April 23	 Redmi 9A Sport (Coral Green, 2GB RAM, 32GB Storage) 2GHz... ★★★★☆ 159,691 ₹6,999.00 Get it by Saturday, April 23	 OnePlus Nord CE 2 5G (Bahamas Blue, 8GB RAM, 128GB Storage) ★★★★☆ 12,639 ₹24,999.00 Get it by Saturday, April 23	 boAt Airdopes 121v2 True Wireless Earbuds with Upto 14... ★★★★☆ 116,391 ₹1,299.00 Get it by Saturday, April 23	 APLUS Super Saver Wheat Flour (Maida) 500 g ★★★★☆ 489 ₹30.00	 Mattel Uno Playing Card Game ★★★★☆ 32,216 ₹104.00 Get it by Saturday, April 23
 Tata Sampann Unpolished Toor Dal/ Arhar Dal, 1kg ★★★★☆ 12,726 ₹135.00 - ₹189.00	 Maggi 2-Minute Noodles Masala, 70g (Pack of 12) ★★★★☆ 47,101 ₹136.00 - ₹330.00	 Fortune Premium Kachi Ghani Pure Mustard Oil, 1tr PET Bottle ★★★★☆ 13,858 ₹205.00 - ₹214.00	 Happilo 100% Natural Premium California Almonds 1kg Value Pack Pouch High Quality... ★★★★☆ 19,151	 Glucon-D Glucon D Instant Energy Health Drink Nimbu Pani - 1kg... ★★★★☆ 4,335 ₹309.00	 boAt Bassheads 100 in Ear Wired Earphones with Mic(Black) ★★★★☆ 317,716 ₹399.00

Predict the folding and 3D structure of protein

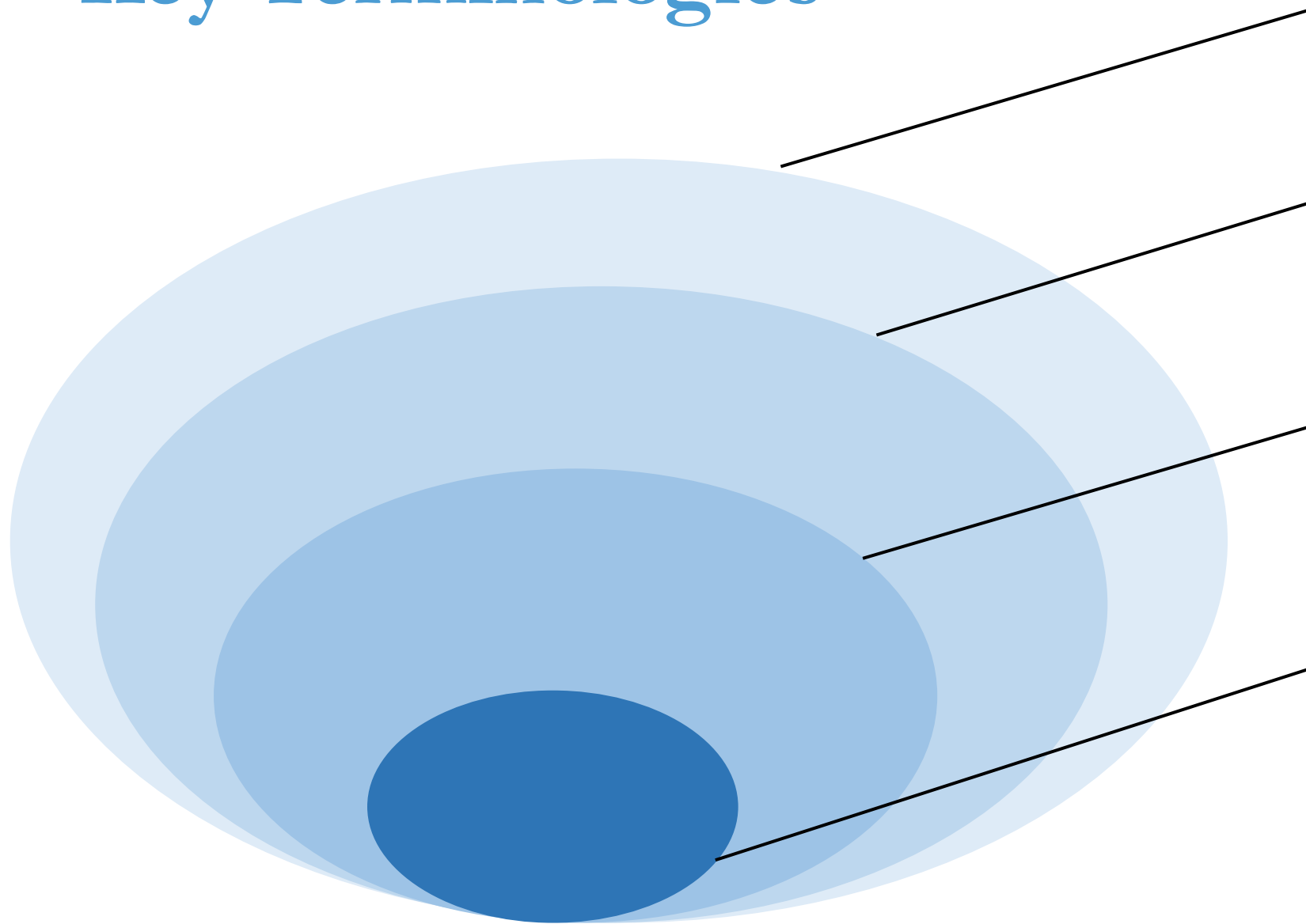
AlphaFold aims to solve the protein folding problem, which involves predicting a protein's three-dimensional structure based solely on its amino acid sequence. Understanding protein structures is crucial for biological research and drug discovery.





What Exactly is Deep Learning?

Key Terminologies



Artificial Intelligence

Simulates human intelligence in machines for tasks like decision-making and language translation.

Machine Learning (ML)

A subset of AI where algorithms learn from data to make predictions or decisions without being explicitly programmed for each scenario.

Deep Learning (DL)

A branch of machine learning using multi-layered neural networks, effective in processing large amounts of unstructured data like images and speech.

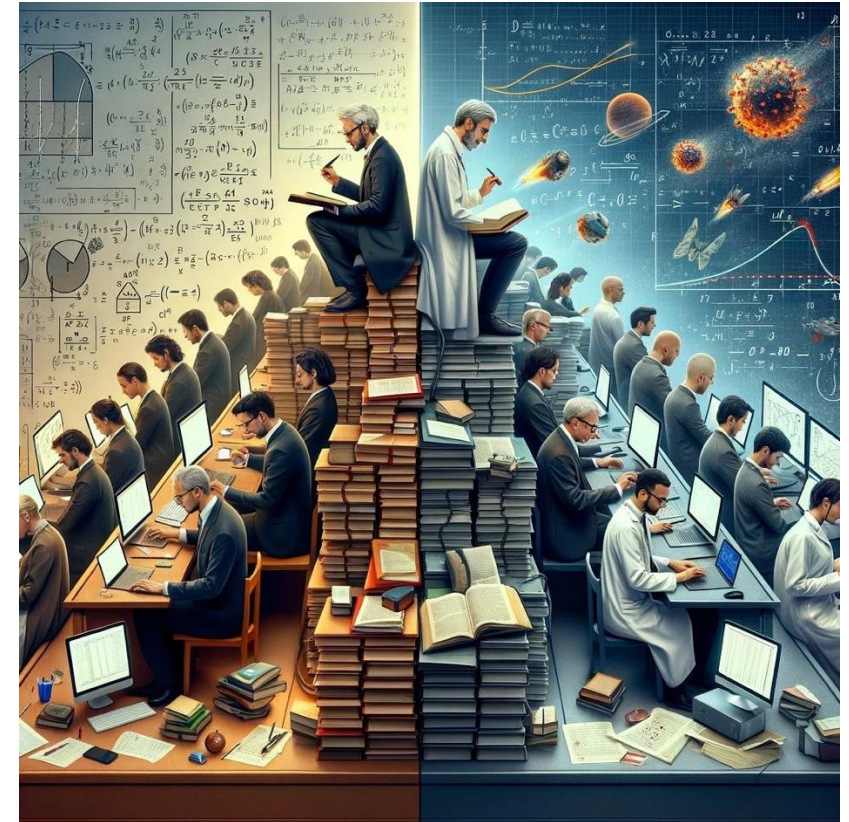
Generative AI

AI algorithms that generate new, original content (like text or images) based on existing data, using techniques like Generative Adversarial Networks (GANs).

Statistical Modeling: The Two Cultures

Statistics is the discipline that concerns the collection, organization, analysis, interpretation, and presentation of data. <https://en.wikipedia.org/wiki/Statistics>

Leo Breiman (2001). *Statistical Science*. There are two cultures in the use of statistical modeling to reach conclusions from data. One assumes that the data are generated by a given stochastic data model. The other uses algorithmic models and treats the data mechanism as unknown. The statistical community has been committed to the almost exclusive use of **data models**. This commitment has led to **irrelevant theory**, **questionable conclusions**, and has kept statisticians from working on a large range of interesting current problems. **Algorithmic modeling**, both in theory and practice, has developed rapidly in **fields outside statistics**. It can be used both on large complex data sets and as a more accurate and informative alternative to data modeling on smaller data sets. *If our goal as a field is to use data to solve problems, then we need to move away from exclusive dependence on data models and adopt a more diverse set of tools.*"



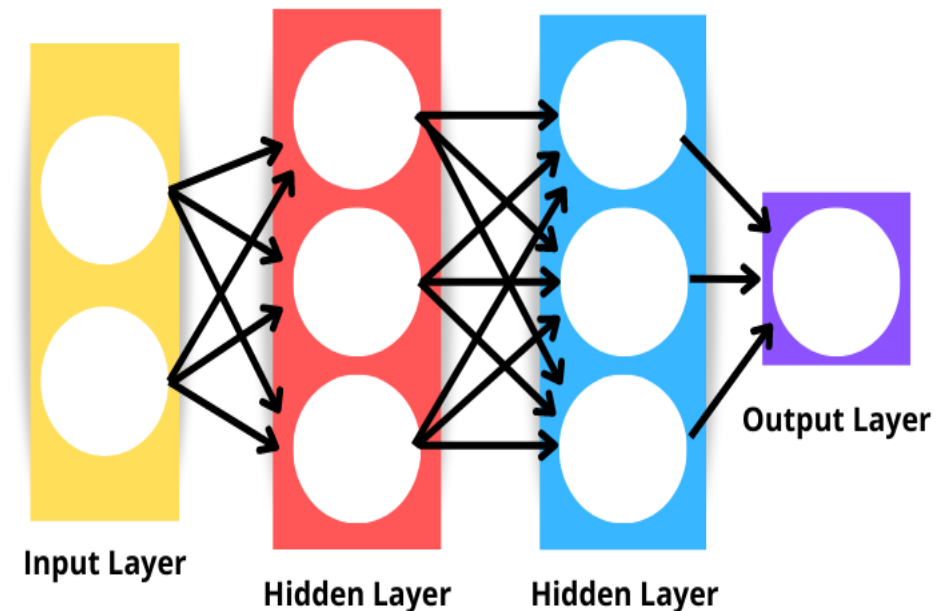
Deep Learning

- Deep learning is a subset of machine learning that focuses on training **algorithmic neural networks** to perform tasks. Its algorithms were inspired by the working of the human brain.
- It's characterized by the use of **multiple layers (deep architectures)** that allow networks to learn hierarchical representations of data and to learn to complete specific tasks.
- In contrast to traditional machine learning/data models, which often requires **manual feature extraction**, deep learning can **automatically learn features from raw data**, which you can think of as patterns that occur within the data.
- Deep learning can be used for supervised, unsupervised, self-supervised, semi-supervised, generative, contrastive, few-shot, as well as reinforcement learning.

Objective: teaching computer how to **learn a task directly from raw data**

Backbone of DL - Neural Networks

- Neural networks, also called artificial neural networks (ANNs) or simulated neural networks (SNNs), are a **subset of machine learning** and are the **backbone of deep learning algorithms**.
- The neural network is inspired by the human brain's interconnected neurons. They are called “neural” because they mimic how neurons in the brain signal one another.
- It consists of layers: an **input layer**, one or more **hidden layers**, and an **output layer**.
- The “**deep**” in deep learning refers to the depth of layers in a neural network.
- Usually, a neural network of more than three layers, including the inputs and the output, can be considered a deep-learning algorithm.



Further details on neural networks will be in upcoming courses.

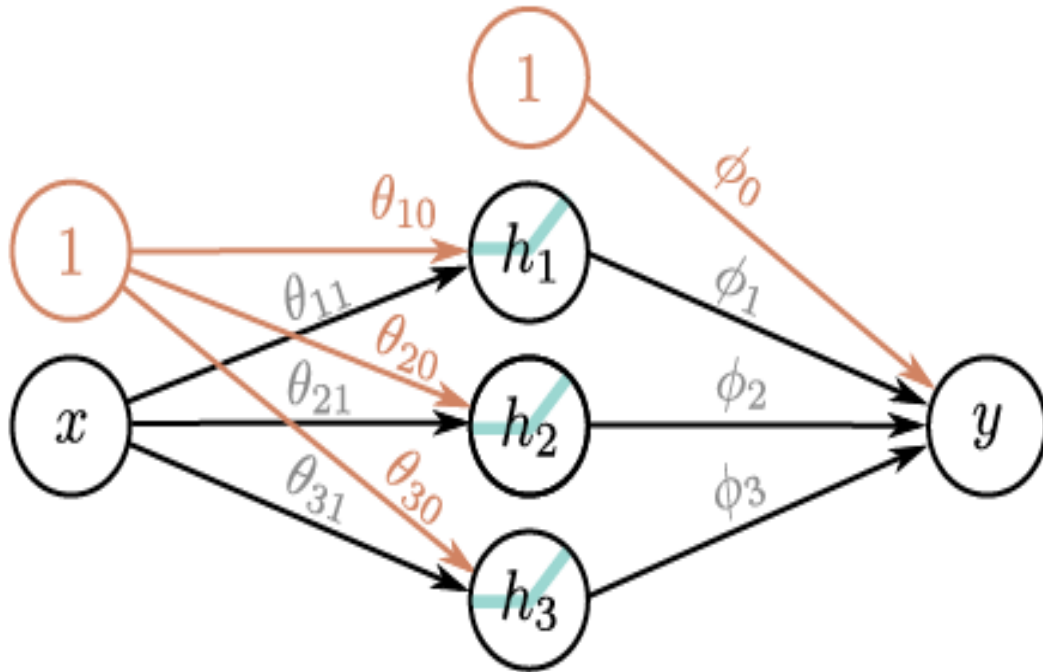
Deep Learning Basics

Neurons (Nodes) receive input signals and perform computations and produce an output.

Channels (connections) are associated with a **weight** value that determines the strength of the connection.

Bias is conceptually similar to the intercept in linear regression, accounting for potential deviations from the ideal relationship between inputs and outputs.

Activation function are threshold values that introduce non-linearities into the neural network, determining if the particular neuron will get activated or not.

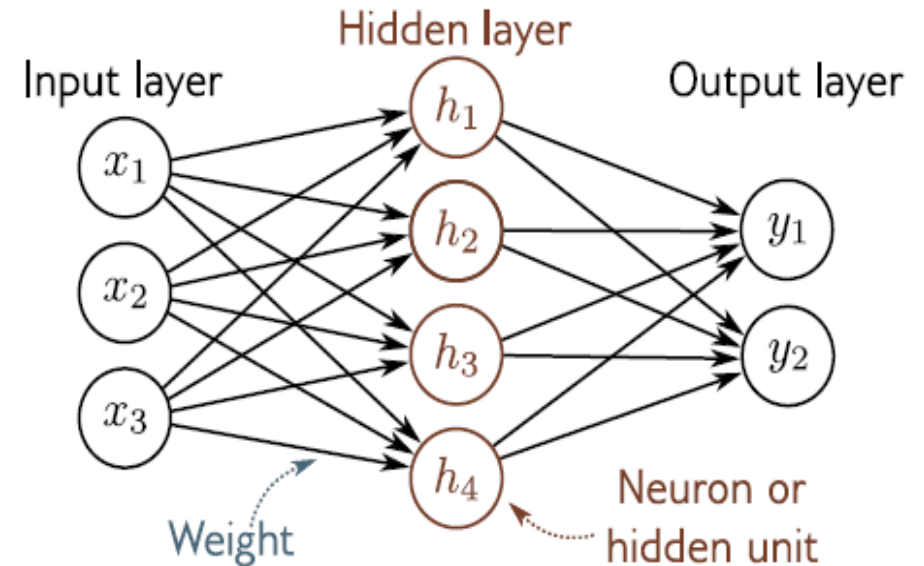


Shallow Neural Network

Universal Approximation Theorem

Cybenko (1989) and Hornik (1991)

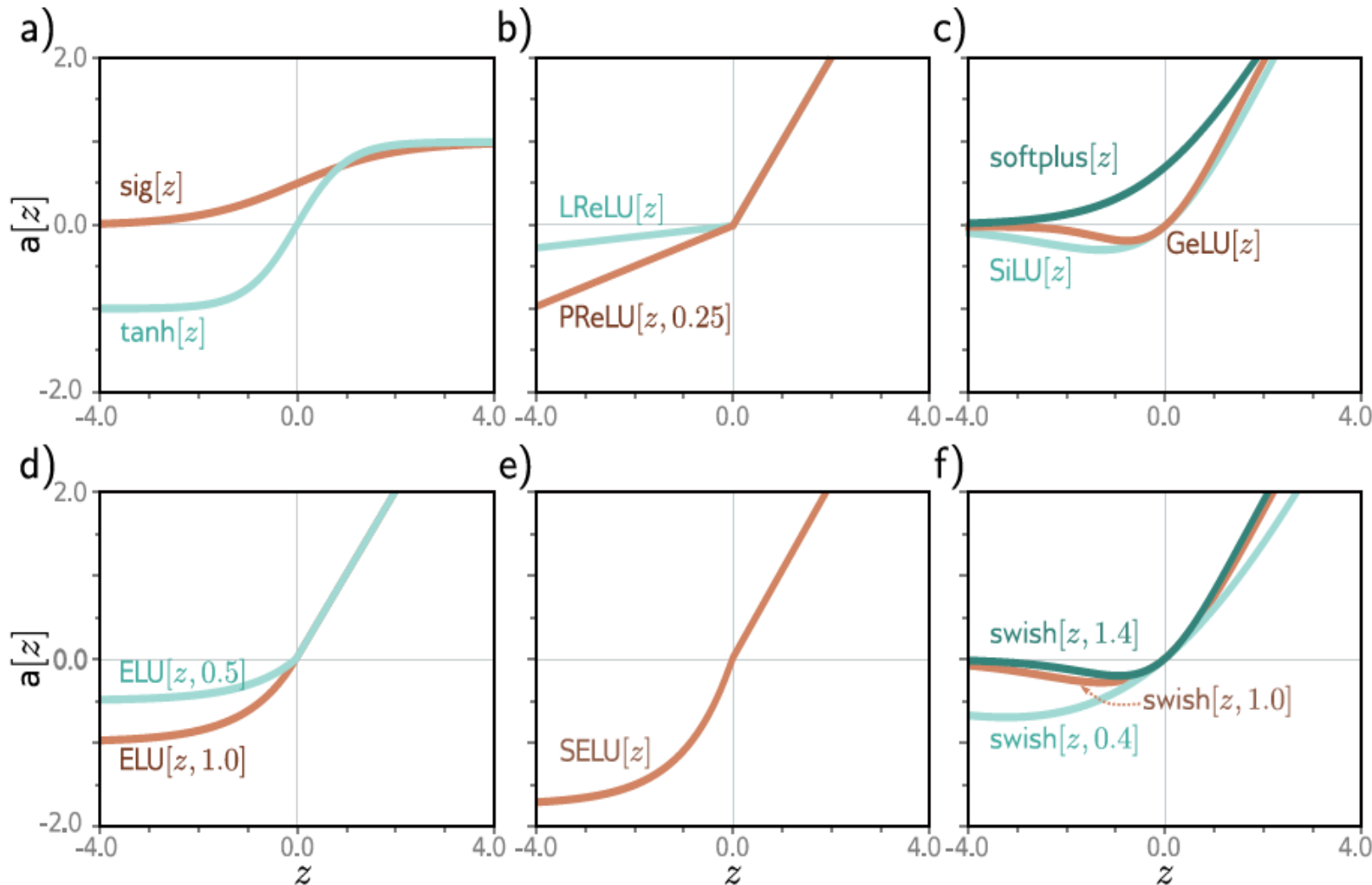
A feed-forward network with a **single hidden layer** containing a finite number of neurons can approximate continuous functions on compact subsets of $\mathbb{R}^n$, under mild assumptions on the activation function.



$$h_d = a \left[\theta_{d0} + \sum_{i=1}^{D_i} \theta_{di} x_i \right],$$

$$y_j = \phi_{j0} + \sum_{d=1}^D \phi_{jd} h_d,$$

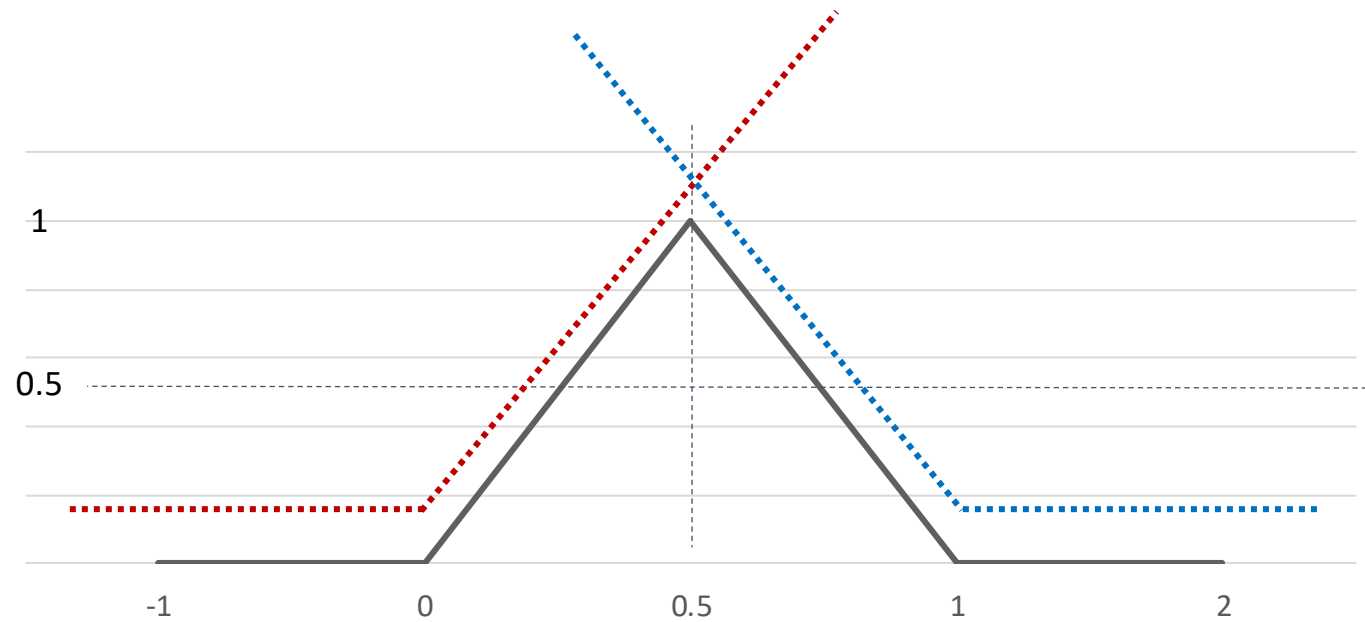
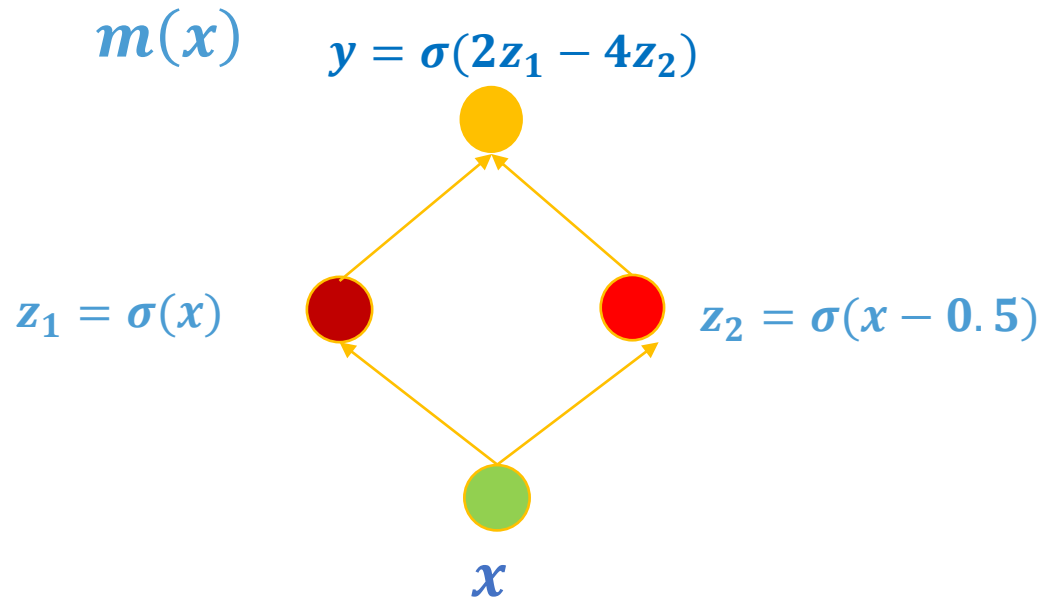
Activation Functions



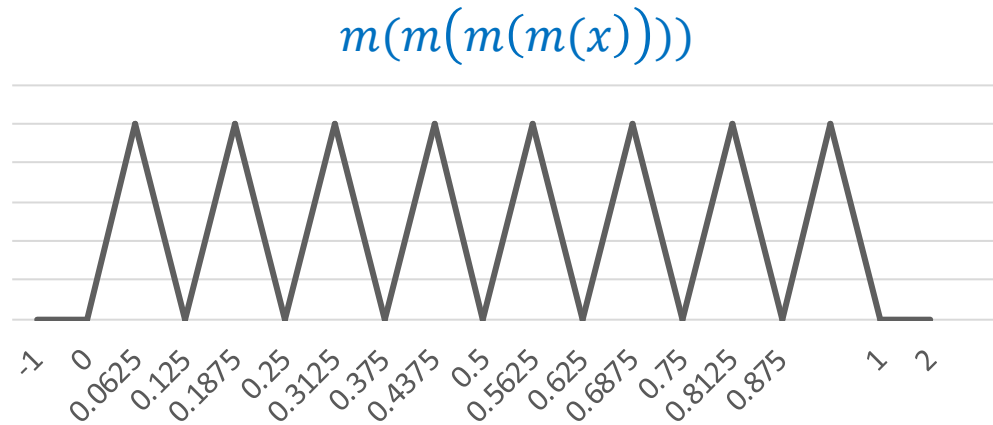
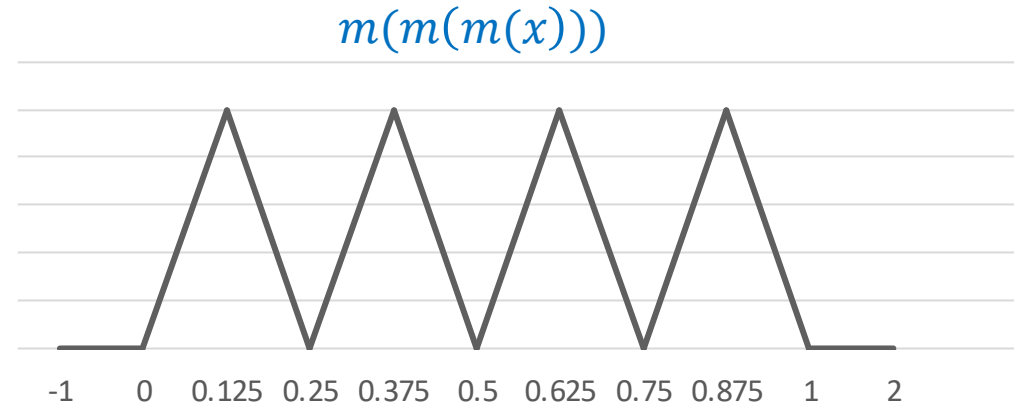
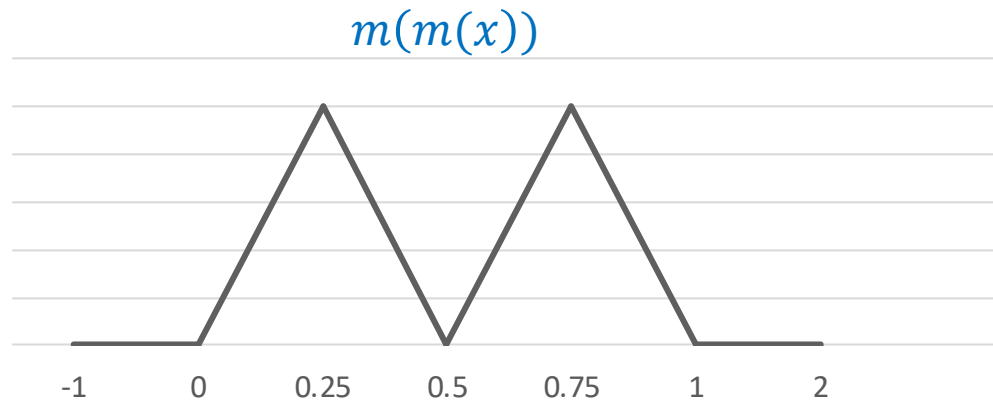
- a) Logistic sigmoid and tanh functions.
- b) Leaky ReLU and parametric ReLU with parameter 0.25.
- c) SoftPlus, Gaussian error linear unit, and sigmoid linear unit.
- d) Exponential linear unit with parameters 0.5 and 1.0.
- e) Scaled exponential linear unit.
- f) Swish with parameters 0.4, 1.0, and 1.4.

Motivation for Deep Learning

Consider a piecewise linear function $m(x) = \begin{cases} 2x, & x \in [0, 0.5] \\ 2 - 2x, & x \in [0.5, 1] \\ 0 & \text{otherwise} \end{cases}$ Define $\sigma(x) = \max(0, x)$

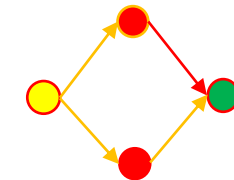


Motivation for Deep Learning



Generate a deep network:

$3n+1$ nodes to represent the $m^{(n)}(x)$
with width of each layer ≤ 2

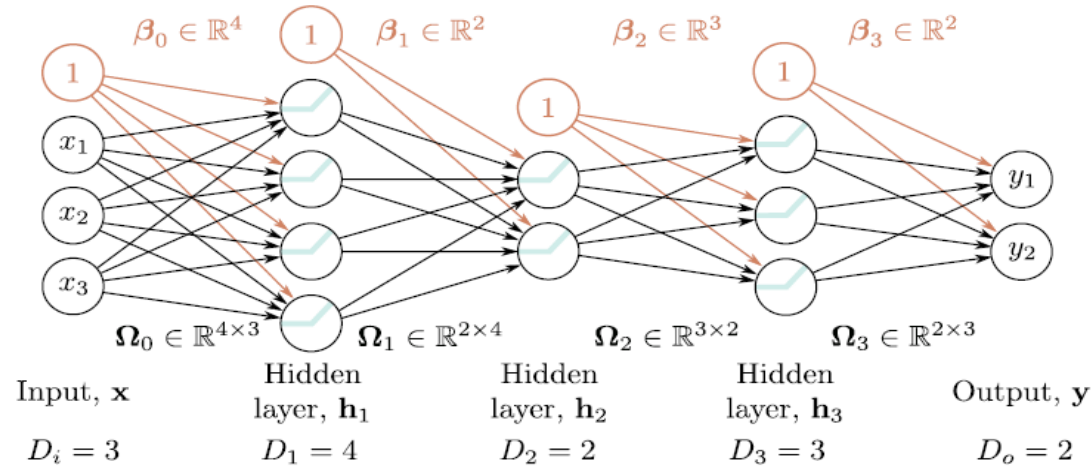


Depth
- Multiplicatively

If we generate a shallow one, we need 2^n nodes

Width
- additively

Deep Neural Network



Shallow vs deep networks

- ❖ both networks can approximate any function given enough capacity,
- ❖ deep networks produce many more linear regions per parameter,
- ❖ some functions can be approximated much more efficiently by deep networks,
- ❖ in practice, the best results for most tasks are achieved using deep networks with many layers.

$$\mathbf{y} = \beta_K + \Omega_K \mathbf{a} [\beta_{K-1} + \Omega_{K-1} \mathbf{a} [\dots \beta_2 + \Omega_2 \mathbf{a} [\beta_1 + \Omega_1 \mathbf{a} [\beta_0 + \Omega_0 \mathbf{x}]] \dots]] .$$

- ❖ The number of hidden units in each layer is referred to as the **width** of the network, and the number of hidden layers as the **depth**. The total number of hidden units is a measure of the network's **capacity**.
- ❖ The **depth version** of the universal approximation theorem (Lu et al., 2017): There exists a network with ReLU activation functions and at least D_i+4 hidden units in each layer can approximate any specified D_i -dimensional Lebesgue integrable function to arbitrary accuracy given enough layers.

$$\begin{aligned} \mathbf{h}_1 &= \mathbf{a}[\beta_0 + \Omega_0 \mathbf{x}] \\ \mathbf{h}_2 &= \mathbf{a}[\beta_1 + \Omega_1 \mathbf{h}_1] \\ \mathbf{h}_3 &= \mathbf{a}[\beta_2 + \Omega_2 \mathbf{h}_2] \\ &\vdots \\ \mathbf{h}_K &= \mathbf{a}[\beta_{K-1} + \Omega_{K-1} \mathbf{h}_{K-1}] \\ \mathbf{y} &= \beta_K + \Omega_K \mathbf{h}_K. \end{aligned}$$

Fitting DL Models

AS=Applied Statistics

Define a set of functions/models



Find a criterion/measurement of goodness –
loss(+ regularization)



Get the best model for the problem

AI=Artificial Intelligence

Design the neural network

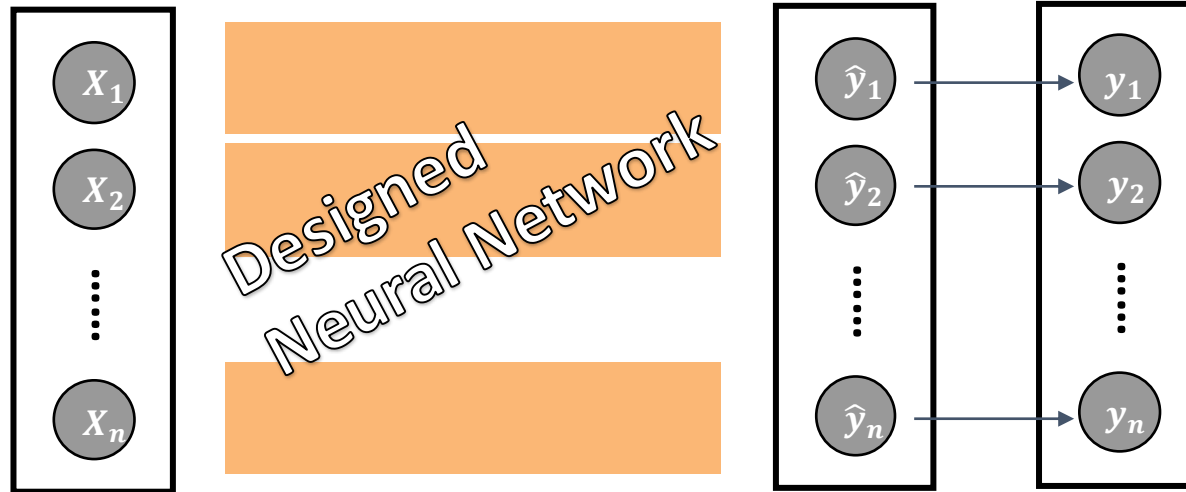


Find a criterion/measurement of goodness –
loss(+ regularization)



Get the best model for the problem

Fitting DL Models



$$\begin{aligned}
 h_1 &= a[\beta_0 + \Omega_0 \mathbf{x}] \\
 h_2 &= a[\beta_1 + \Omega_1 h_1] \\
 h_3 &= a[\beta_2 + \Omega_2 h_2] \\
 &\vdots \\
 h_K &= a[\beta_{K-1} + \Omega_{K-1} h_{K-1}] \\
 y &= \beta_K + \Omega_K h_K.
 \end{aligned}$$

A **loss function** is needed here,
to measure the difference between the output and truth

Total loss:
$$L = \sum \ell(\hat{\mathbf{y}}_i, \mathbf{y}_i)$$

$$\hat{\mathbf{y}}_i = \beta_K + \Omega_K \mathbf{h}_K(\mathbf{x}_i; [(\beta_0, \Omega_0), \dots, (\beta_{K-1}, \Omega_{K-1})])$$

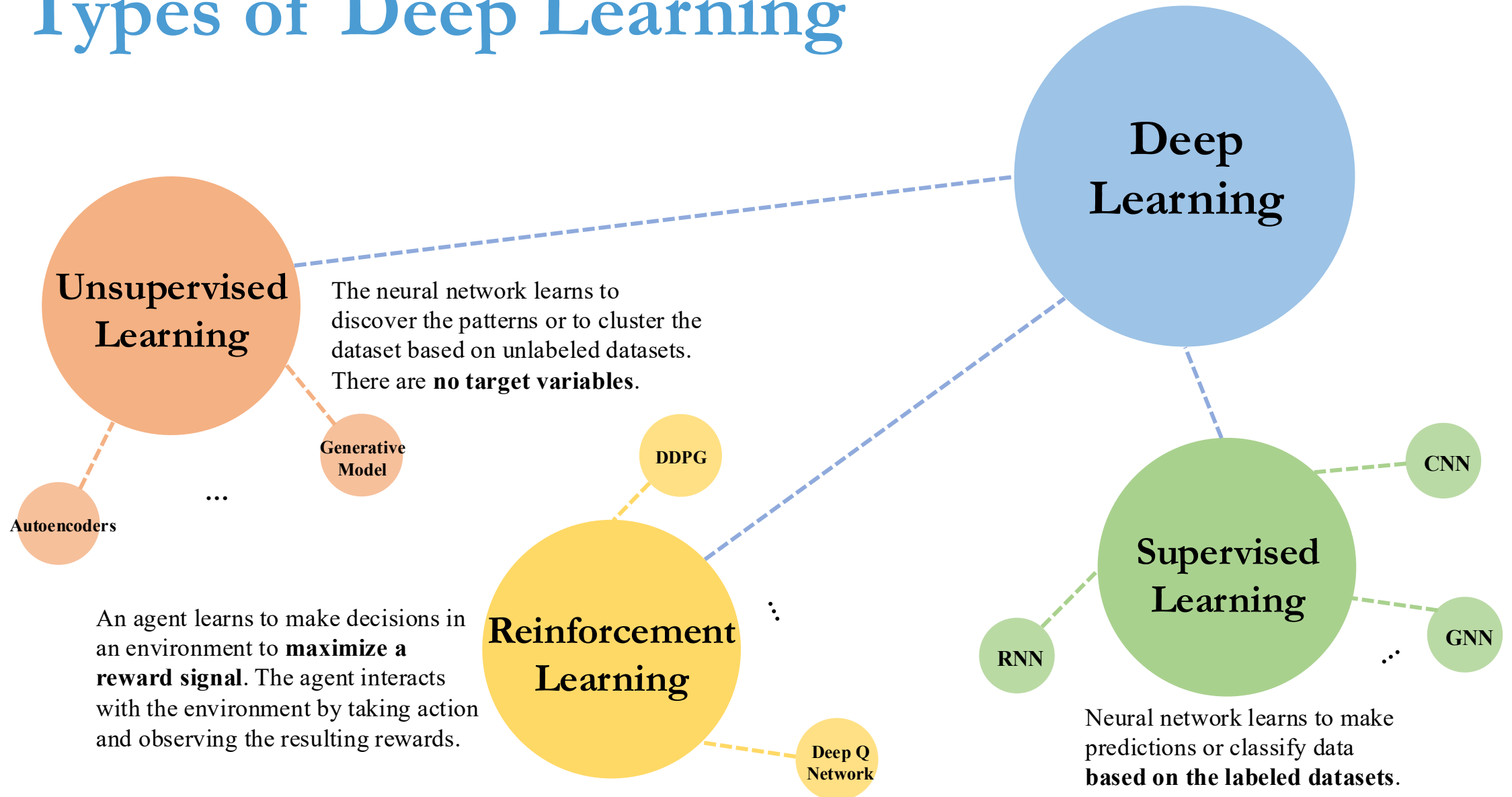
Find the network parameters to minimize the loss

Design the neural network

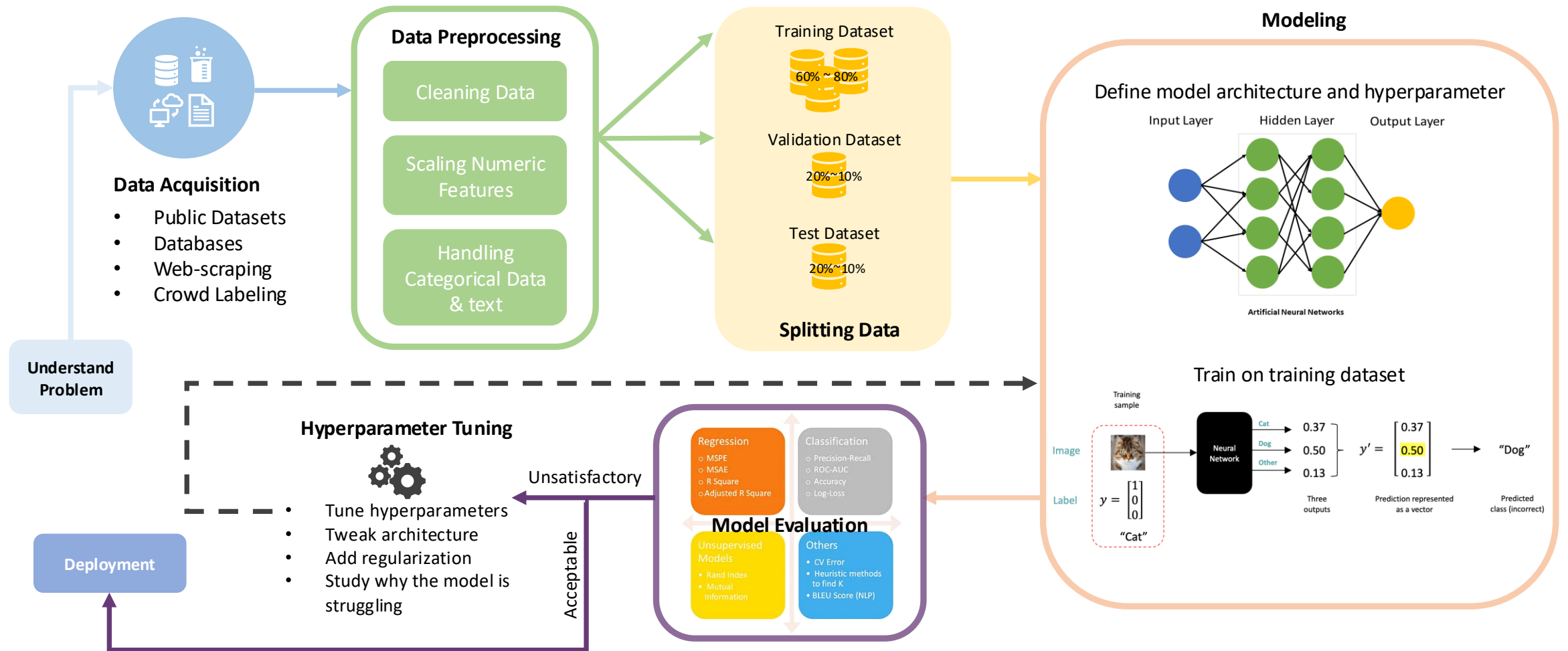
Find a criterion/measurement of goodness

Get the best model for the problem

Types of Deep Learning



Workflow of a Typical DL Project

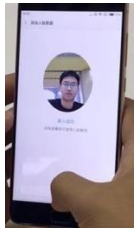


Face Recognition System

Image Acquisition



Camera



Iphone



Monitor Videos

Face Recognition

Face Detection



Feature Extraction



Face Matching



Recognition Results

Model Training



Gabor, LBP etc.



Deep Learning



Skill Sets

Computer Science

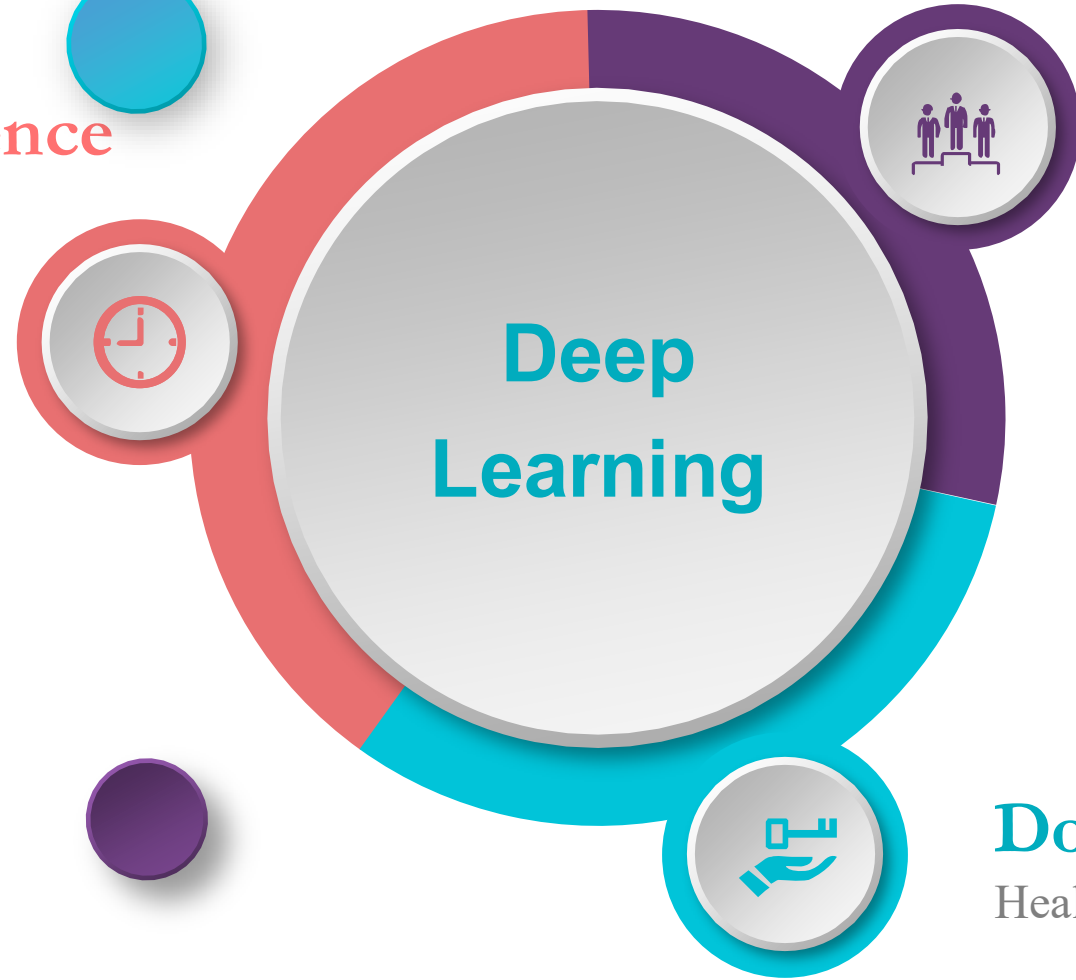
Programming

Math & Statistics

Probability, Linear algebra,
Statistical modeling

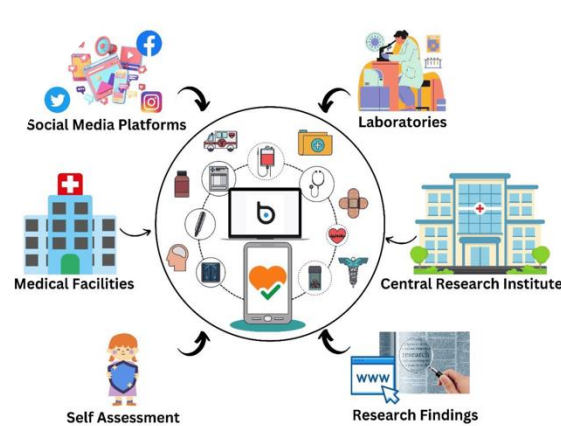
Domain Knowledge

Health science, medical



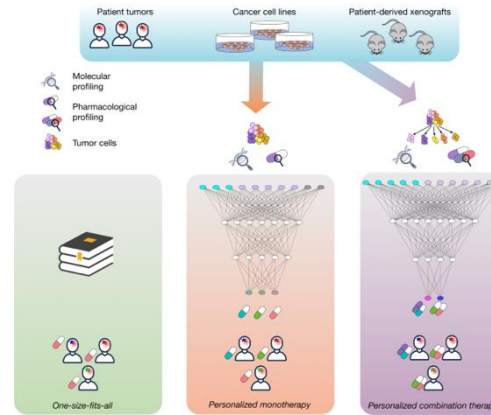
Deep
Learning

Some Future Directions in DL for Biostatistics



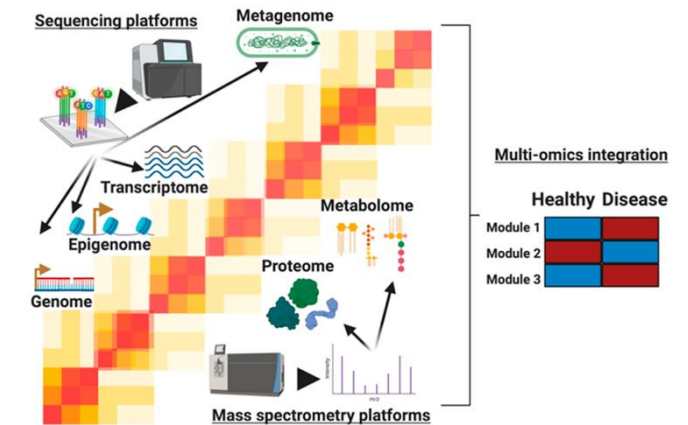
AI-driven Public Health Interventions

Utilizing deep learning models to analyze large-scale public health data for informed decision-making and policy development. Allowing better resource allocation, and more effective epidemic control strategies.



Advanced Drug Response Modeling

Using deep learning to model and predict individual responses to drugs, considering genetic, environmental, and lifestyle factors. Developing more effective personalized treatments.



Integrative Analysis of Multi-omic Data

Leveraging deep learning to integrate and analyze data from genomics, proteomics, metabolomics, and other omics fields for a comprehensive understanding of biological processes and disease mechanisms.

Generalist Medical Artificial Intelligence

- **Foundation Models in Medicine:** These models leverage large-scale datasets and generalizable architectures to address diverse medical tasks, moving beyond task-specific AI systems.
- **Generalist AI:** Unlike traditional models, foundation models aim to function across multiple domains, such as imaging, text, and genomics, enabling integration of multimodal data for holistic medical insights.
- **Challenges:**
 - Data heterogeneity: Medical data comes in varied formats, requiring harmonization.
 - Privacy and ethics: Ensuring secure, unbiased AI while maintaining patient confidentiality.
 - Interpretability: Providing clinicians with actionable insights from AI outputs.
- **Applications:**
 - Diagnostics: Detecting diseases across imaging modalities (e.g., radiology).
 - Prognostics: Predicting patient outcomes using integrated data.
 - Personalized medicine: Tailoring treatments based on multimodal patient profiles.
- **Future Directions:**
 - Collaboration between AI experts and clinicians to co-design models.
 - Development of robust validation frameworks for clinical adoption.
 - Advancing explainability and trust in AI-driven medical decisions.

Moor, M.,, Rajpurkar, P. (2023) *Nature*.

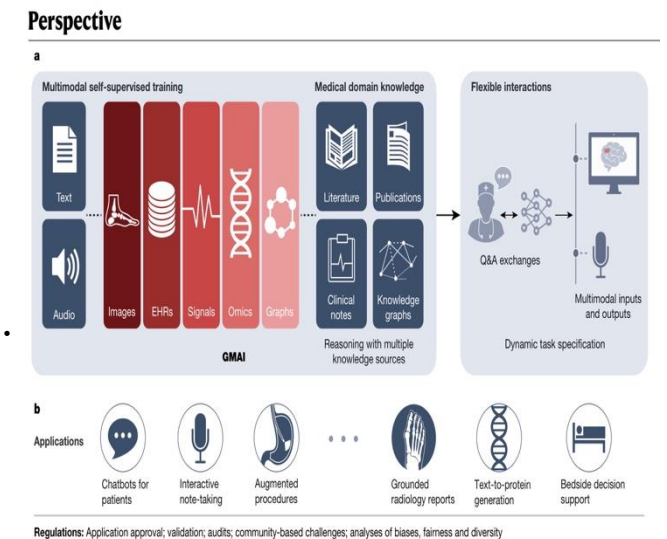


Fig. 1 | Overview of a GMAI model pipeline. **a.** A GMAI model is trained on multiple medical data modalities, through techniques such as self-supervised learning. To enable flexible interactions, data modalities such as images or data from EHRs can be paired with language, either in the form of text or speech data. Next, the GMAI model needs to access various sources of medical knowledge to carry out medical reasoning tasks, unlocking a wealth of capabilities that can be used in downstream applications. The resulting GMAI model then carries out tasks that the user can specify in real time. For this, the GMAI model can retrieve contextual information from sources such as knowledge graphs or databases, leveraging formal medical knowledge to reason about previously unseen tasks. **b.** The GMAI model builds the foundation for numerous applications across clinical disciplines, each requiring careful validation and regulatory assessment.

Content

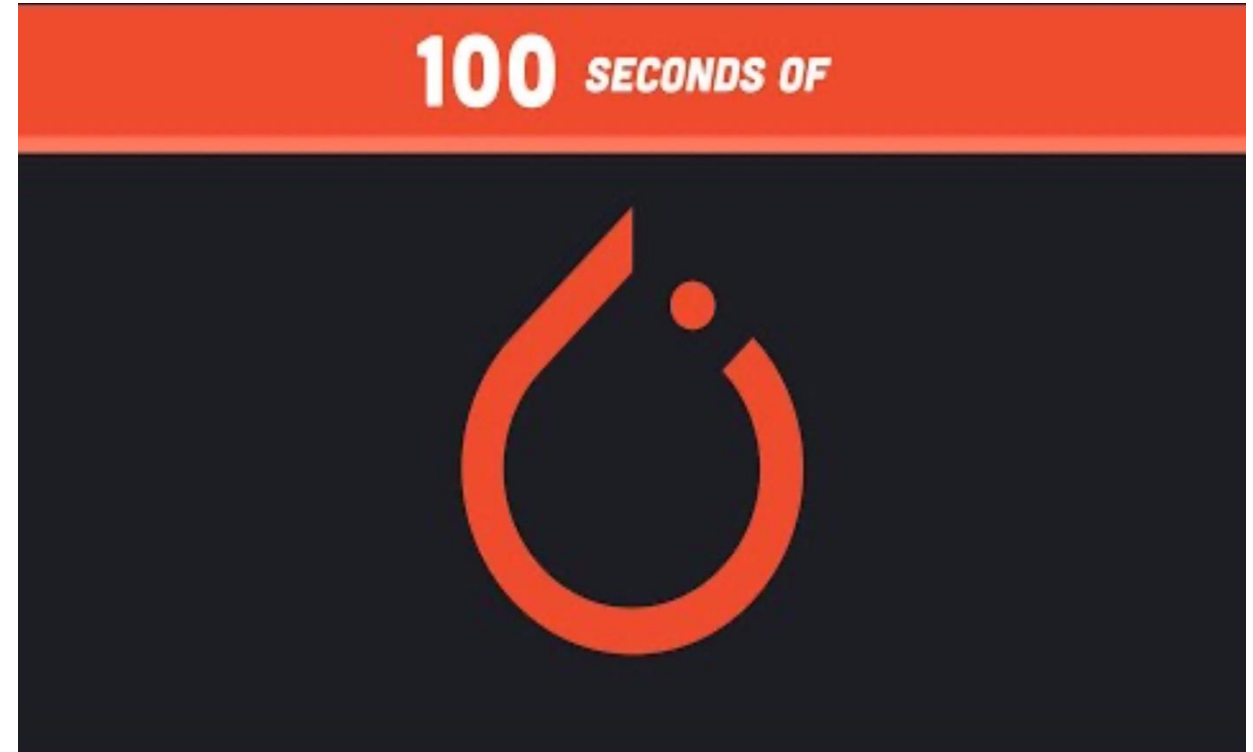
1 Introduction to Deep Learning

2 Introduction to PyTorch

3 Introduction to UNC Research Computing Resources

4 Introduction to Basic Algorithm

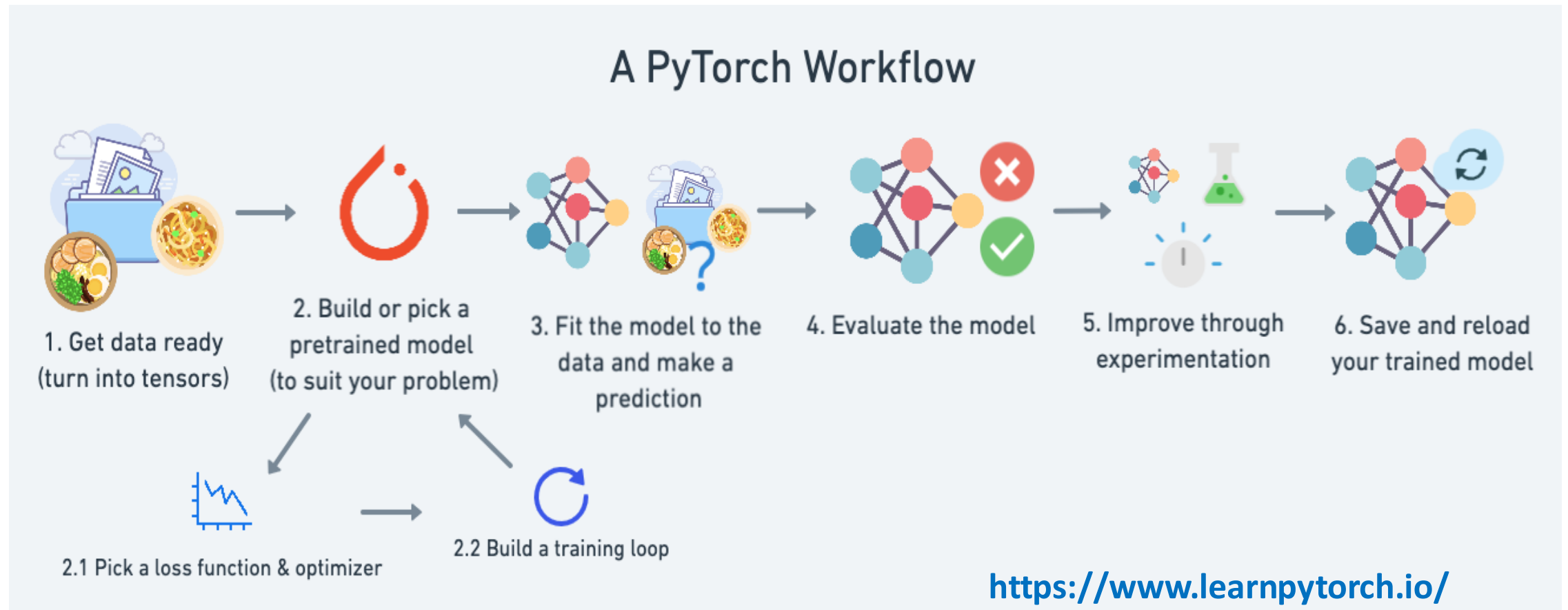
Why Torch?



What is PyTorch?

- **PyTorch**, originally built at **Facebook (now Meta)**, is a Python-based machine learning library designed to provide flexibility and efficiency for developing deep learning models. It is widely used in academia and industry due to its intuitive and dynamic design.
<https://pytorch.org/tutorials/beginner/basics/intro.html>
- **Key Features:**
 - **Dynamic Computation Graphs:** Modify the model's architecture during runtime, making it easier to debug and experiment.
 - **GPU Acceleration:** Seamlessly integrate GPU computations for speedup.
 - **Extensive Ecosystem:** Includes libraries such as **torchvision** (computer vision), **torchtext** (NLP), and **torchaudio** (audio processing).
 - **Integration with Python:** PyTorch operates natively in Python, allowing access to Python libraries and tools.
- **Common Use Cases:**
 - Building neural networks for image recognition, natural language processing, and generative models.
 - Research and experimentation due to flexibility in designing and debugging models.
 - Production-level deployment using tools like TorchScript and TorchServe.

A PyTorch Workflow

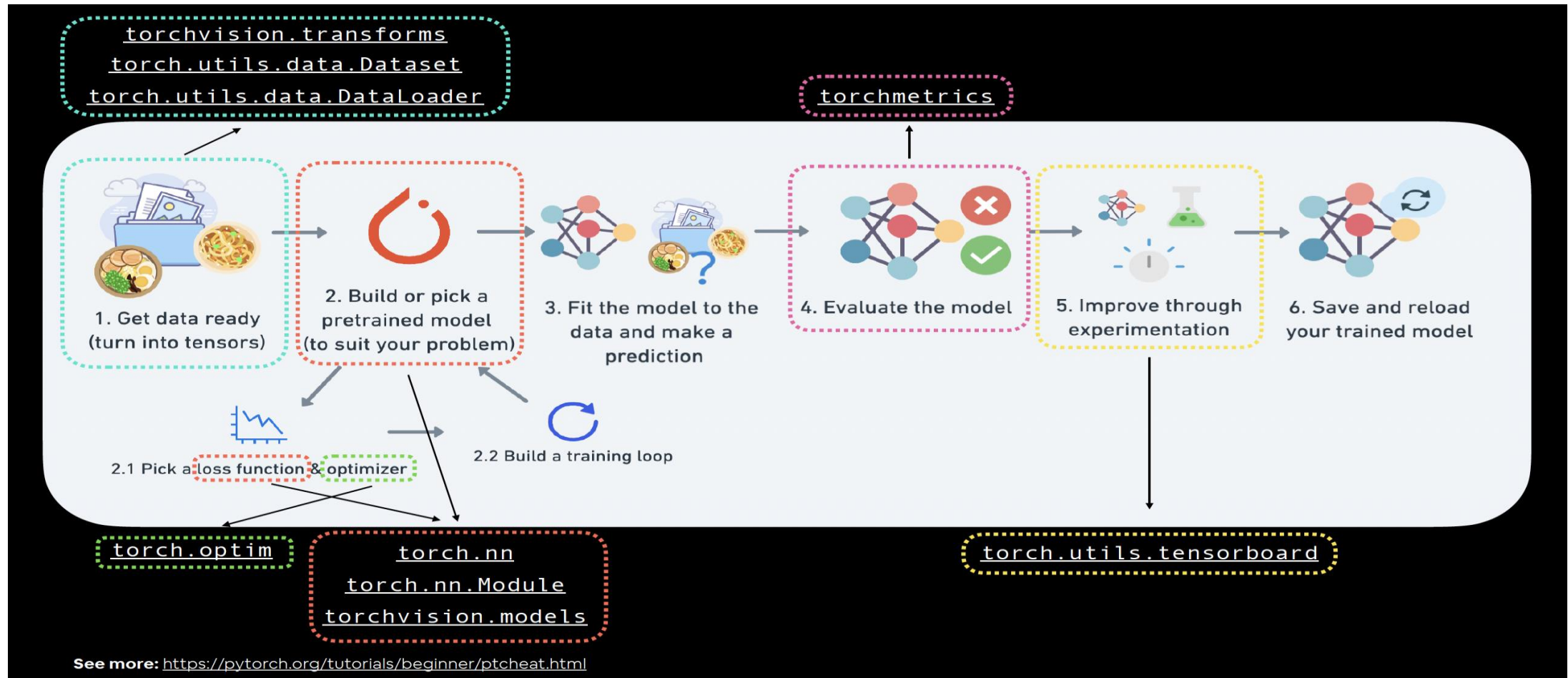


PyTorch essential building modules

PyTorch module	What does it do?
<u><code>torch.nn</code></u>	Contains all of the building blocks for computational graphs (essentially a series of computations executed in a particular way).
<u><code>torch.nn.Module</code></u>	The base class for all neural network modules, all the building blocks for neural networks are subclasses. If you're building a neural network in PyTorch, your models should subclass <code>nn.Module</code> . Requires a <code>forward()</code> method be implemented.
<u><code>torch.optim</code></u>	Contains various optimization algorithms (these tell the model parameters stored in <u><code>nn.Parameter</code></u> how to best change to improve gradient descent and in turn reduce the loss).
<u><code>torch.utils.data.Dataset</code></u>	Represents a map between key (label) and sample (features) pairs of your data. Such as images and their associated labels.
<u><code>torch.utils.data.DataLoader</code></u>	Creates a Python iterable over a torch Dataset (allows you to iterate over your data).

See more: <https://pytorch.org/tutorials/beginner/ptcheat.html>

A PyTorch Workflow with Modules



<https://pytorch.org/tutorials/>

PyTorch Example: Model Definition

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import torch.optim as optim

# 1) Define a simple MLP
class Net(nn.Module):
    def __init__(self, input_dim=320, hidden_dim=50, num_classes=10, dropout_p=0.5):
        super(Net, self).__init__()
        self.fc1 = nn.Linear(input_dim, hidden_dim)
        self.fc2 = nn.Linear(hidden_dim, num_classes)
        self.dropout_p = dropout_p

    def forward(self, x):
        x = F.relu(self.fc1(x))
        x = F.dropout(x, p=self.dropout_p, training=self.training)
        x = self.fc2(x)
        return x

# 2) Instantiate the model
net = Net()
```

What this does

- Defines a two-layer MLP for 10-class classification
- Uses ReLU + Dropout for nonlinearity and regularization
- Returns logits (use CrossEntropyLoss)

PyTorch Example: Data + Training Loop

```
# 3) Dummy dataset + DataLoader
X = torch.randn(1000, 320)
y = torch.randint(0, 10, (1000,))
train_dataset = torch.utils.data.TensorDataset(X, y)
train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=32, shuffle=True
)

# 4) Loss + optimizer
criterion = nn.CrossEntropyLoss()
optimizer = optim.SGD(net.parameters(), lr=0.001, momentum=0.9)

# 5) Training loop
net.train()
for epoch in range(10):
    for inputs, labels in train_loader:
        optimizer.zero_grad()
        outputs = net(inputs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()
```

Key ideas

- DataLoader batches + shuffles data each epoch
- CrossEntropyLoss expects logits and integer labels
- SGD updates weights using gradients from backprop
- net.train() enables Dropout during training

Working with Tensors

- Tensors are the foundation of PyTorch, enabling efficient numerical computations. They are similar to NumPy arrays but optimized for GPUs. Tensors are used to encode inputs and model weights.
- **Creating Tensors:**
 - Scalars, vectors, matrices, and higher-dimensional tensors:

Numpy style operations

```
tensor = torch.ones(4, 4)
print(f"First row: {tensor[0]}")
print(f"First column: {tensor[:, 0]}")
print(f"Last column: {tensor[:, -1]}")
tensor[:, 1] = 0
print(tensor)
```

Out:

```
First row: tensor([1., 1., 1., 1.])
First column: tensor([1., 1., 1., 1.])
Last column: tensor([1., 1., 1., 1.])
tensor([[1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.],
        [1., 0., 1., 1.]])
```

Capability of CPU/GPU computing

```
# We move our tensor to the GPU if available
if torch.cuda.is_available():
    tensor = tensor.to("cuda")
x = torch.tensor([1.0]).to('cuda') # Move to GPU
y = x.to('cpu') # Move back to CPU
```

Common Operations:

- Element-wise operations: +, -, *, /.
- Matrix operations: torch.matmul, torch.mm.
- Reshaping: .reshape(), .squeeze(), .unsqueeze().

Datasets and DataLoader

PyTorch provides a structured approach to handling datasets through Dataset and DataLoader.

Dataset:

The Dataset class allows you to define how data samples are accessed and prepared.

Custom datasets can be implemented by subclassing `torch.utils.data.Dataset` and defining `__getitem__` and `__len__` methods.

Built-in Datasets: PyTorch provides ready-to-use datasets such as MNIST and CIFAR-10 through `torchvision.datasets`.

DataLoader:

- Combines datasets into batches, shuffles data, and handles multiprocessing for loading data efficiently.

Loading FashionMNIST

```
from torchvision import datasets
training_data = datasets.FashionMNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor())
```

Building a DataLoader

```
from torch.utils.data import DataLoader

train_dataloader = DataLoader(training_data, batch_size=64,
                               shuffle=True)
train_features, train_labels = next(iter(train_dataloader))
```

In practice, many researches opt to write their own data processing and sampling code to gain greater flexibility.

Network Structure

- The neural network is defined by subclassing `nn.Module`, the layers are initialized in `__init__` and the operations on input data is in forward method.
- After instantize the network, the network do forward by passing the input data.

```
class NeuralNetwork(nn.Module):
    def __init__(self):
        super().__init__()
        self.flatten = nn.Flatten()
        self.linear_relu_stack = nn.Sequential(
            nn.Linear(28*28, 512),
            nn.ReLU(),
            nn.Linear(512, 512),
            nn.ReLU(),
            nn.Linear(512, 10),
        )

    def forward(self, x):
        x = self.flatten(x)
        logits = self.linear_relu_stack(x)
        return logits
```

Key Components:

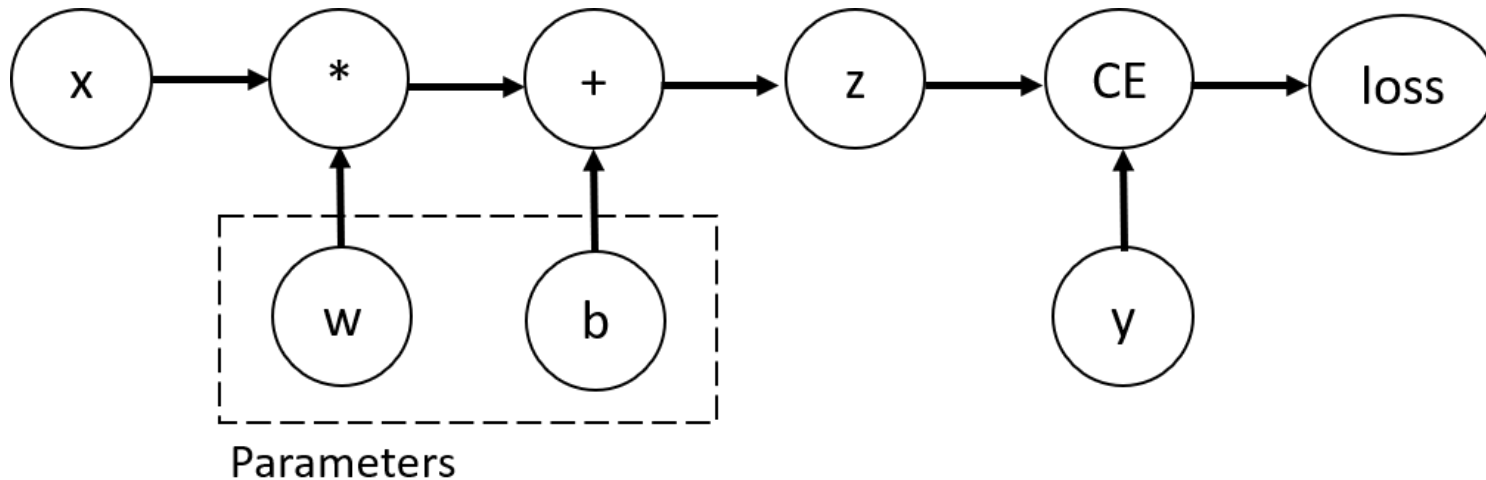
- **Modules:** Predefined layers like `nn.Linear`, `nn.Conv2d`, `nn.LSTM`.
- **Sequential Models:** Simplify model definition:
- **Custom Models:** Subclass `nn.Module` to define custom architectures:
- **Activation Functions:** PyTorch provides various activation functions such as ReLU, Sigmoid, and Tanh.

Instantize and send weights to GPU

```
model = NeuralNetwork().to(device)
X = torch.rand(1, 28, 28, device=device)
logits = model(X)
```

Auto-differentiation

- When training neural networks, the most frequently used algorithm is gradient descent (and its variants). Pytorch has a built-in engine called autograd for calculating gradients.



```
x = torch.ones(5)  # input tensor
y = torch.zeros(3) # expected output
w = torch.randn(5, 3, requires_grad=True)
b = torch.randn(3, requires_grad=True)
z = torch.matmul(x, w)+b
loss = torch.nn.functional.binary_cross_entropy_with_logits(z, y)
```

- Pytorch have the notion of computation graph.
- Each time do calculations, pytorch store the graph structure (how the final output is related to the tensors that require grad).
- When you use backward to calculate the grad, pytorch calculates the gradient of each component that requires_grad and store it.

Optimization

- Optimization algorithms define how model parameters adjust to reduce model error in each training step.
- Many optimization algorithms (e.g. Adam) are available in torch.optim.

```
# Define the optimizer
optimizer = torch.optim.SGD(model.parameters(), lr=learning_rate)

# Within each training step, first reset the gradient of parameters
# Then calculate the gradients of the loss w.r.t. each parameter.
optimizer.zero_grad()

# Adjust the parameters using gradients
optimizer.step()
```

Training and Testing Loops

Training a neural network involves iterative optimization to minimize a loss function.

- **Steps in Training:**

- **Initialize the Model:** Define the network and its parameters.
- **Define the Loss Function:** Choose a loss function (e.g., nn.MSELoss, nn.CrossEntropyLoss).
- **Select an Optimizer:** Use optimizers like torch.optim.SGD or torch.optim.Adam.

- **Training Loop:**

```
for epoch in range(n_epochs):  
    model.train() # Set model to training mode  
    for x, y in dataloader:  
        optimizer.zero_grad() # Reset gradients  
        output = model(x) # Forward pass  
        loss = criterion(output, y) # Compute loss  
        loss.backward() # Backward pass  
        optimizer.step() # Update weights
```

Testing Loop: Use .eval() mode and disable gradient calculations:

```
model.eval()  
with torch.no_grad():  
    for x, y in test_loader:  
        output = model(x)
```

Content

1 Introduction to Deep Learning

2 Introduction to PyTorch

3 Introduction to UNC Research Computing Resources

4 Introduction to Basic Algorithm

UNC Longleaf

Before begin:

- Establish a VPN connection with UNC.
 - [For VPN instructions, search <https://help.unc.edu> for VPN.]
- If you do not have longleaf account yet, request it now:
 - <https://help.rc.unc.edu/request-a-cluster-account/>
- *If there's time, download...*
 - *and install the ssh app longleaf prefers:*
 - <https://help.rc.unc.edu/getting-logged-on/>
 - *These Using Longleaf course slides:*
 - <https://its.unc.edu/research-computing/research-computing-presentations/>

UNC ITS Computing Resources

Provides "computing infrastructure as well as other technology tools and capabilities to support the research needs of University faculty and staff"

Resource examples:

- VCL: Virtual Computing Lab
- **Longleaf Cluster (today)**
- Dogwood Cluster
- DGX Cluster
- Secure Research Workstation
- Access to Xsede Campus Champions program

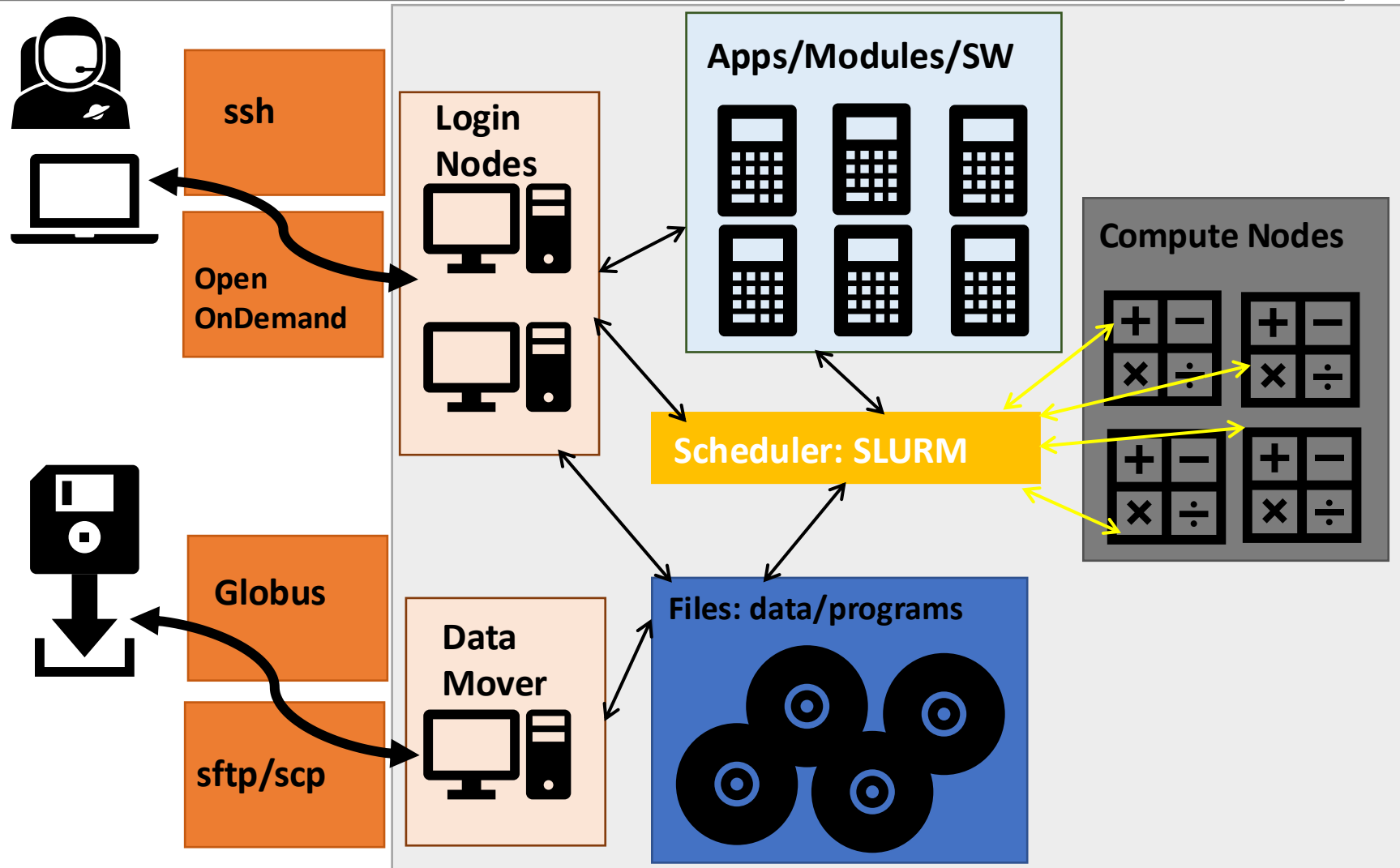
List grows all the time...including some cloud computing.

Need help?

<https://help.rc.unc.edu>

research@unc.edu answered during business hours, M-F

Longleaf (LL) Components



Before Computing on Longleaf

<https://island-climb-ea6.notion.site/Longleaf-GPU-guidance-1723d297a55f803daccff5e46f77659e>

Properly

- **Connect** to Longleaf (log on)
- **Move** your project's data & programs to/from storage areas in longleaf
- Pick and **manage** your software “modules”
- **Schedule** jobs to run on longleaf's compute nodes
- **Monitor** the status of jobs, their output and results
- Navigate a Linux operating system (*not in this course*)

THEN you can have fun doing your project on longleaf.

Access Longleaf

Two ways to connect:

1. Via an **ssh connection**: `ssh -X <onyen>@longleaf.unc.edu`

*Requires an ssh program on your computer, and **not** the one your computer came with.*

If connecting from off-campus, your computer is required to establish a VPN connection to UNC before connecting to longleaf. Search help.unc.edu for VPN.

2. Via **Open OnDemand (OOD)**: <https://ondemand.rc.unc.edu>

****New in 2020****

Yes any browser will do! (But not your cell phone)

Limited access to longleaf, but enough for many users.

DUO authentication required.

Read More: <https://help.rc.unc.edu/getting-logged-on/>

Content

1 Introduction to Deep Learning

2 Introduction to PyTorch

3 Introduction to UNC Research Computing Resources

4 Introduction to Basic Algorithm

Case Study: Logistic Regression in PyTorch

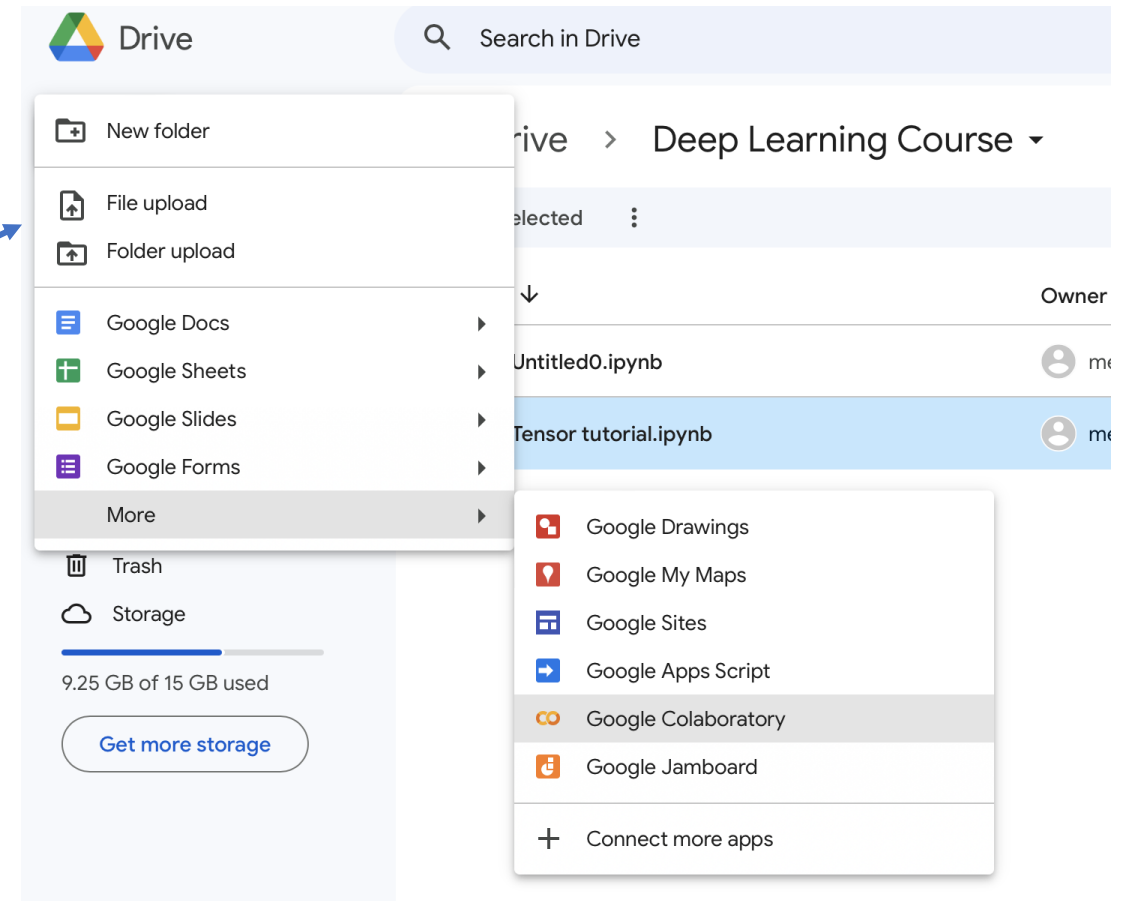
Identifying handwritten digits using Logistic Regression in PyTorch.ipynb

Case Study: Logistic Regression in PyTorch

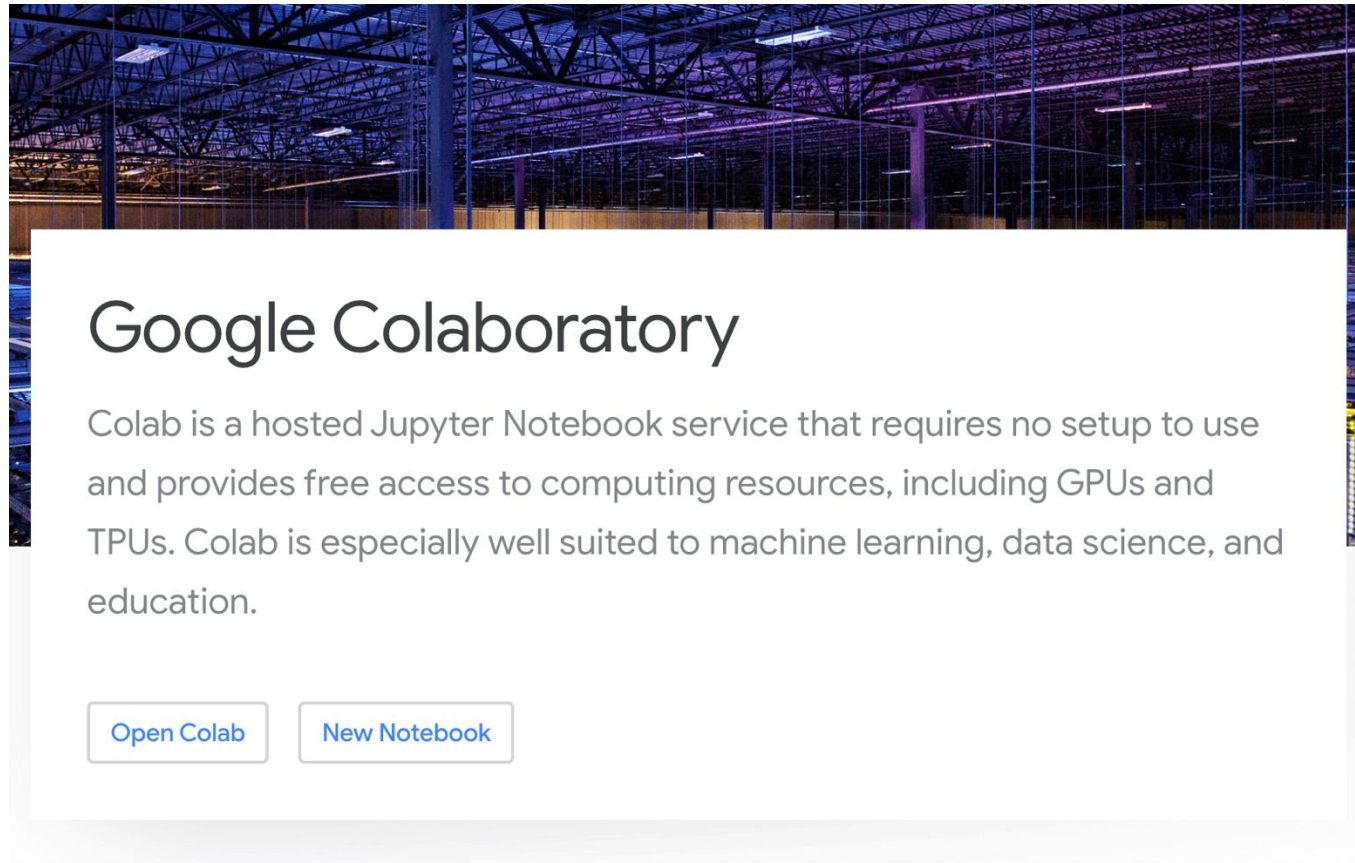
Running example code in Google Colab – a good start for beginners

First step: How to access to Google Colab?

1. Log into Google Drive.
2. In Google Drive, create a new Google Colab notebook. 
3. Start coding then.



Google Colab

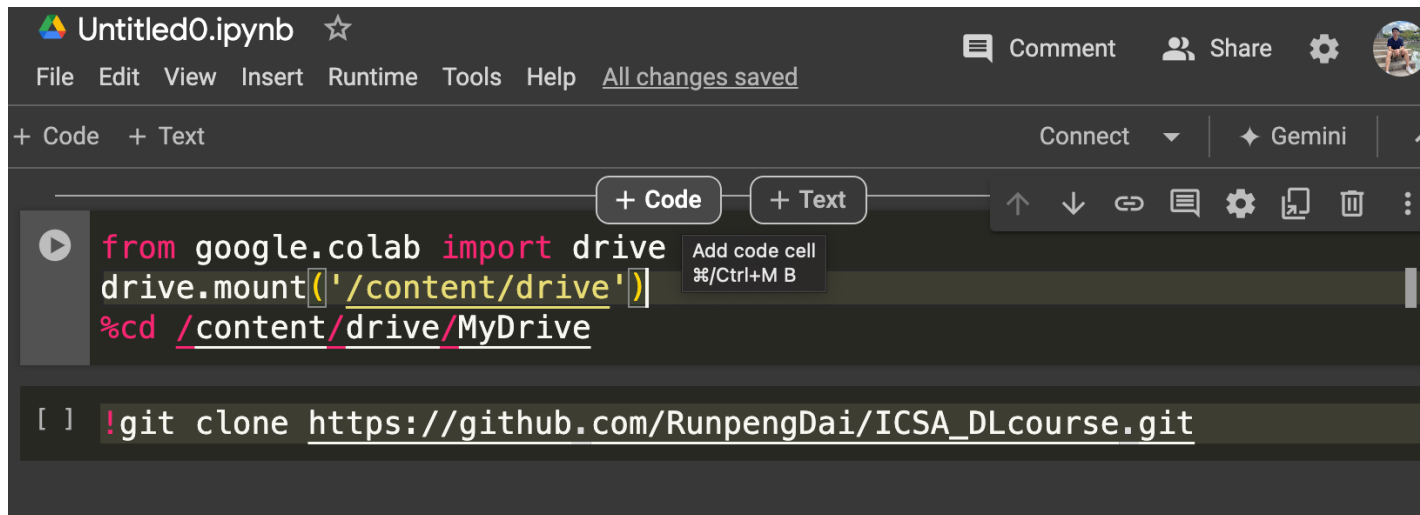


- Google Colab, short for Colab, is a free, cloud-based platform provided by Google Research. It allows users to write and execute Python code through a web browser.
- One of Colab's key features is the use of Jupyter Notebooks
- Colab provides free access to powerful hardware accelerators, including GPUs and TPUs
- Colab can mount google drive.

Setup

Google Colaboratory

Colab is a hosted Jupyter Notebook service that requires no setup to use and provides free access to computing resources, including GPUs and TPUs. Colab is especially well suited to machine learning, data science, and education.



```
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/MyDrive

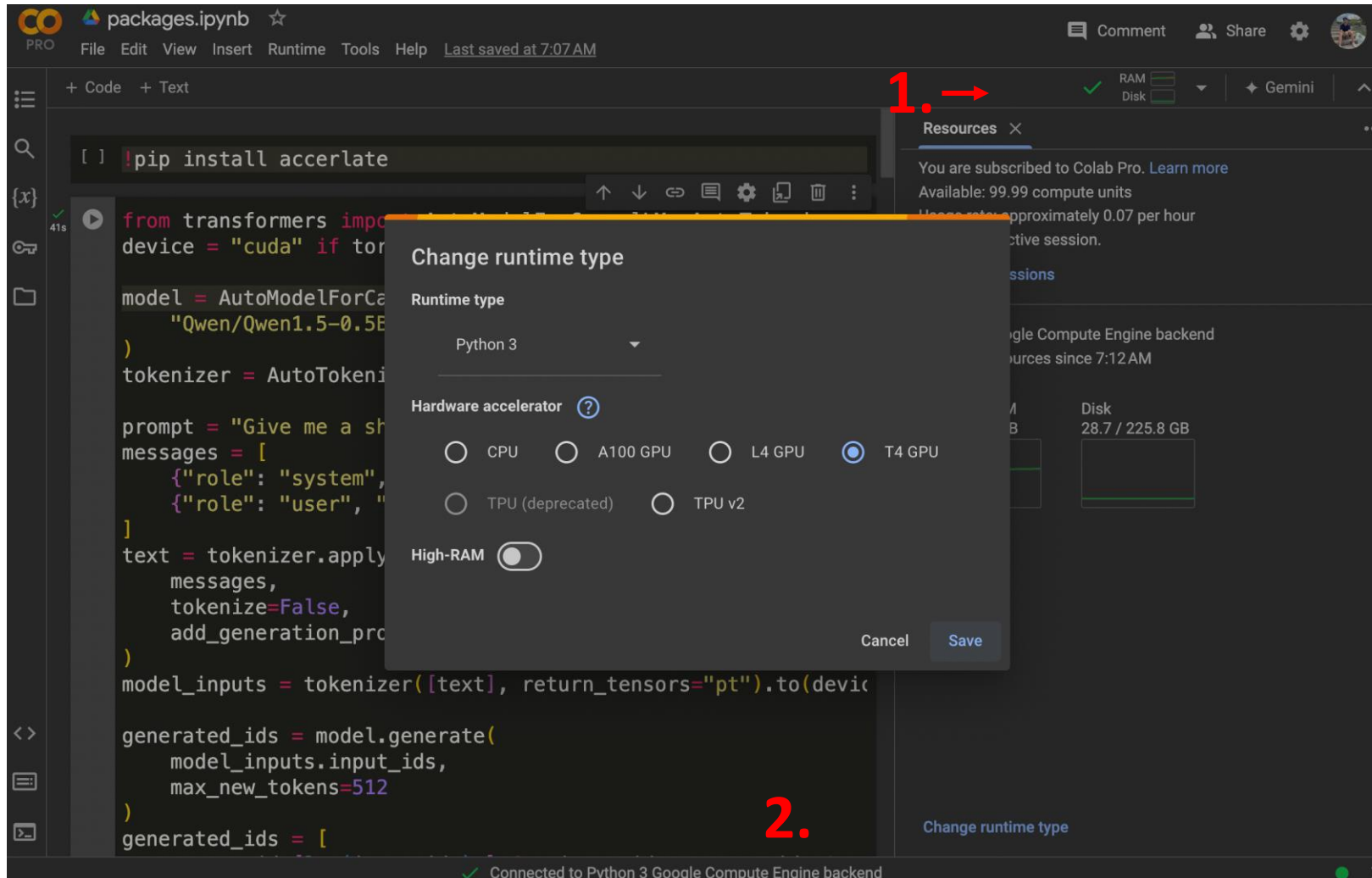
[ ] !git clone https://github.com/RunpengDai/ICSA_DLcourse.git
```

```
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/MyDrive
```

```
!git clone
https://github.com/RunpengDai/ICSA_DLcourse.git
```

- We offer multiple coding sessions in the short course.
- We are going to clone the git repo into Google drive using commands in colab.
- First, create a empty Notebook through <https://colab.google/>.
- Then execute the following code, to mount Google drive to colab and clone our repo to Google drive.

Code session – Pytorch Basics



- First open packages.ipynb from the intro folder within Google drive.
- Change the runtime type to T4 GPU to have access to GPU computing resource.

References



<https://github.com/mrdbourke/pytorch-deep-learning>



<https://docs.pytorch.org/tutorials/>



Goodfellow, Ian, Yoshua Bengio, Aaron Courville, and Yoshua Bengio. *Deep learning*. Vol. 1, no. 2. Cambridge: MIT press, 2016.



LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *nature*, 521(7553), 436-444.

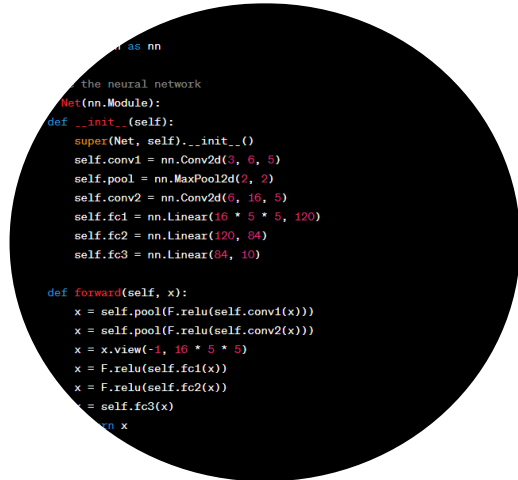


Pearson, Helen, Heidi Ledford, Matthew Hutson, and Richard Van Noorden. "Exclusive: the most-cited papers of the twenty-first century." *Nature* 640, no. 8059 (2025): 588-592.



Binson, V. A., Sania Thomas, Monikavasagom Subramoniam, Jayaseelan Arun, Subbaiyan Naveen, and S. Madhu. "A review of machine learning algorithms for biomedical applications." *Annals of Biomedical Engineering* 52, no. 5 (2024): 1159-1183.

How to succeed in this course?



Practice



Explore



Visualize

Discuss



Ask

